

环境驾驭：唤醒静态世界以赋能智能体学习

Chengsong Huang^{1*}, Zifeng Wang², Rujun Han², Jun Yan², Yanfei Chen², Zoey CuiZhu², Ke Jiang², Peng Xia⁴, Han Yu², Yufan Zhuang², Yifei Ming², Jiaqi Pan³, Bhavana Dalvi Mishra², Jiaxin Huang¹, Burak Gokturk², Tomas Pfister² and Chen-Yu Lee²

¹Washington University in St. Louis, ²Google Cloud AI Research, ³Google Cloud, ⁴University of North Carolina at Chapel Hill

大语言模型智能体通过与环境的交互进行学习，然而这些环境往往是人工构建且静态不变的：它们无法感知智能体的弱点，并随着智能体的进步而迅速被超越。尽管近期环境生成方法试图解决这一问题，但它们需要特定领域的流程，依赖昂贵或不可靠的验证器，并且仍然生成静态环境。为了减轻从头重建环境的工程负担，本文提出环境驾驭（Environment Harness，简称 EnvHarness），这是一个由可插拔组件构成的可编程层，它包裹静态环境，在不修改底层逻辑的前提下重塑其行为。EnvHarness 通过标准接口运行，可跨多个领域应用，同时确保每个重塑后的环境保留其原始验证器。为了实现该过程的自动化，本文引入 EnvRigger，它将目标策略视为黑盒，通过观察其执行轨迹来合成针对已诊断缺陷的 EnvHarness 组件，并通过新的运行轨迹进行验证。在四个领域的五个基准测试中，EnvHarness 均优于原始环境和特定领域的环境生成流程，在留出实例上实现了高达 9.0 分的提升，同时执行步骤减少了 9.8%。此外，EnvHarness 为强化学习提供了更优的优化信号，实现了策略与环境之间持续、有针对性的协同进化。

github.com/google-research/envharness www.envharness.com

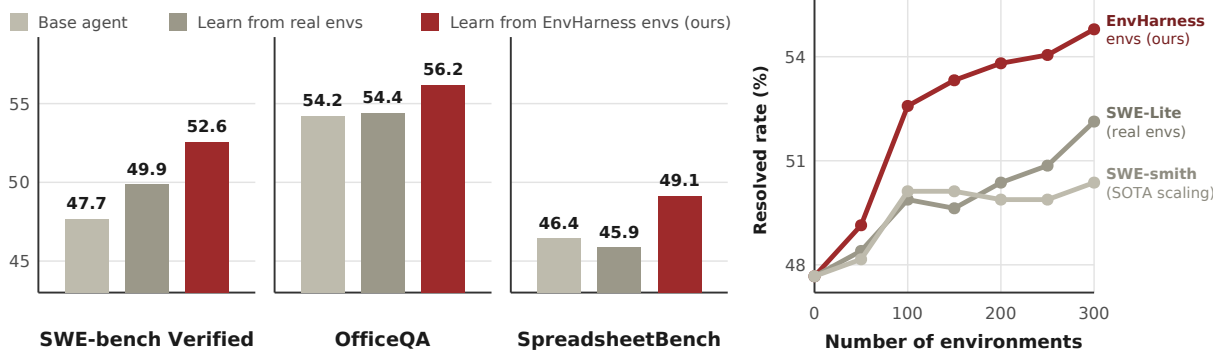


图 1 | 总体性能。左图：在软件工程和办公自动化基准测试（包括 SWE-bench Verified、OfficeQA 和 SpreadsheetBench）中，从 EnvHarness 环境学习的智能体始终优于从原始环境学习的智能体。右图：在 SWE-bench Verified 上，在相同的环境预算下，EnvHarness 随着环境规模的扩大持续改进，而真实环境和生成环境则趋于平缓。

1. 引言

随着大语言模型被部署为自主智能体，学习的来源从精心整理的文本数据转向交互式环境。无论是浏览网页（Gur 等人，2024）、解决

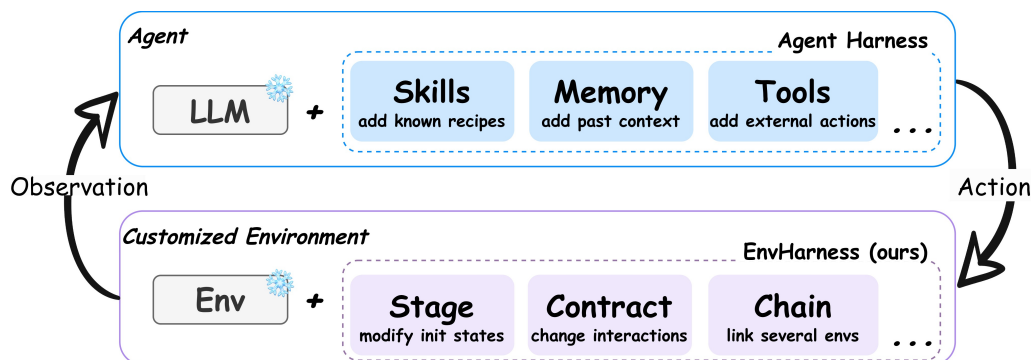


图 2 | 智能体框架 (Agent Harness) 通过插件组件 (如技能、记忆、工具) 将冻结的大语言模型 (LLM) 转化为具备能力的智能体, 而无需改变模型权重; 环境框架 (EnvHarness) 则将这一原理应用于交互的另一侧。它通过插件组件定制冻结的环境, 同时保持原始环境不变。

代码库 (Yang 等, 2024), 或控制具身平台 (Wang 等, 2023), 智能体依赖其各自的环境来获取学习信号 (Yao 等, 2022b)。这些环境作为交互对应方, 呈现特定任务、管理变化状态、响应动作并评估成功 (Li 等, 2026)。遗憾的是, 构建这些环境需要大量人力来硬编码交互逻辑和验证器 (Merrill 等, 2026; Zhang 等, 2025)。因此, 生成的环境保持刚性静态, 无论哪个智能体与之交互或该智能体改进多少, 其行为都完全相同 (Hu 等, 2026)。这种刚性在两个基本方面限制了智能体学习: 它无法提供针对特定智能体弱点的定向信号 (Dennis 等, 2020; Jiang 等, 2021), 并且一旦智能体学会解决现有任务, 它就没有更多可教的内容 (Beukman 等, 2024; Wang 等, 2019)。

由于手动构建环境成本高昂, 越来越多的研究转向自动化环境生成 (Guo 等, 2025; Song 等, 2026; Zala 等, 2024)。尽管具有可扩展性, 这种方法仍存在两个主要局限性。首先, 生成流程本质上具有领域特定性。现有系统为网页导航 (Trabucco 等, 2025)、编程 (Pan 等, 2024) 或工具使用 (Lee 等, 2026a) 生成环境, 但为一种场景构建的流程无法迁移到其他场景。其次, 确保正确性既昂贵又不可靠。由于这些环境和验证器由大语言模型生成, 实践者必须过度生成并大量过滤 (Wang 等, 2026; Yang 等, 2026a), 即便如此仍无法完全保证其正确性。

本文提出环境挂载器 (**EnvHarness**), 一种可编程层, 旨在将现有静态环境转换为动态定制环境, 而无需修改环境本身, 从而避免为获取学习信号而创建新环境。图 2 对环境挂载器与智能体挂载器 (Anthropic, 2025, 2026b; OpenAI, 2026) 进行了类比: 智能体挂载器为大型语言模型提供外部记忆、工具和技能, 以处理超出基础文本生成范畴的复杂任务。环境挂载器将这一概念扩展到环境层面, 为静态环境配备模块化、即插即用的组件。阶段 (Stage) 设定回合的起始点, 契约 (Contract) 控制允许的动作和观测, 链 (Chain) 连接多个基础环境以形成扩展回合。智能体继续使用标准接口, 但环境挂载器在其中介导交互。这使得单一环境能够满足其原本设计之外的需求: 隔离特定技能、延长任务时间跨度, 或校准难度以使智能体经历挣扎但最终成功。关键在于, 严格在接口层面操作使环境挂载器具备领域无关性, 同时允许每个新环境安全继承其原始环境中经过人工验证的可信验证器。

尽管环境挂载器提供了普遍适用的框架，但其具体配置必须针对每个目标策略和任务进行定制。为实现这一定制过程的自动化，本文引入环境装配器（EnvRigger）。环境装配器将策略严格视为黑盒，通过观察基础环境中的成功与失败执行轨迹来诊断具体的行为缺陷。基于这些发现，它合成候选的环境挂载器组件，并封装当前环境以对其进行评估。随后，环境装配器执行新的策略展开，仅根据候选环境是否有效培养缺失能力且保持可解性来判断是否接受。未能通过此评估的组件将经历迭代修正，直至成功。最终，该工作流程实现了完全自动化、任务与策略条件化的环境定制。

我们在具有挑战性的基准上进行了实验，涵盖具身任务（ALFWorld（Shridhar 等，2020））、网页浏览（WebArena（Zhou 等，2024））、软件工程（SWE-bench Verified（Jimenez 等，2024））和办公工作（OfficeQA（Databricks，2025）、SpreadsheetBench（Ma 等，2024））。具体来说，我们在两种代表性学习范式下评估 EnvHarness：基于技能的学习（SL）和强化学习（RL）。在 SL 设置中，使用 EnvHarness 环境训练的智能体优于在原始环境中训练的智能体，在留出任务上最多提高 9.0 分（表 2），同时交互步骤减少 9.8%（表 3）。在 RL 设置中，EnvHarness 定制的环境同样产生显著更强的策略，最多提高 6.5 分（表 4）。最后，反复执行 EnvRigger 循环使智能体与其环境之间能够持续共同进化，解锁复合性能增益，并随定制任务数量有效扩展。

我们的贡献有三方面：（1）我们提出 EnvHarness，一个可编程层，通过其自身的重置 / 步骤接口将静态环境定制为可控环境，实例化为三类插件组件，重塑环境初始状态、智能体-环境交互接口以及来自不同环境的复合任务，同时原始环境的任务和验证器保持不变。（2）我们引入 EnvRigger 来自动化任务-策略条件的环境定制。通过从回放中诊断策略缺陷并迭代修订候选组件，直到新的回放确认其成功，EnvRigger 确保每个新环境都针对策略的特定弱点。（3）在四个领域的五个基准上，EnvHarness 提高了有效性（留出任务上最多提高 9.0 分）和效率（步骤减少 9.8%），在强化学习下增强了策略，并在人类构建和生成的环境都趋于平缓时扩展了智能体性能。

2. EnvHarness

2.1. EnvHarness 范式

表 1 | 智能体 Harness 与 EnvHarness 的类比。两者都通过外部层扩展能力，而不是改变核心系统。

	Agent Harness	ENVHARNESS (Ours)
Base System	Frozen LLM	Static environment
Designed to Solve	Lack of action, memory, or loops	Hardcoded interaction logic
Harness Layer	Capabilities (tools, memory)	Customization (states, rules, observations)
Unified Output	Autonomous agent	Customized environment

An *agent harness* (Anthropic, 2025, 2026b; OpenAI, 2026) is the software layer (execution loops, tool registries, and context management) (Meng et al., 2026) that wraps a LLM to form an autonomous agent (Agent = Model + Harness). It adds new capabilities without changing the model weights. We

将相同的思路应用于智能体-环境循环的另一侧。本文将环境挂载器（**EnvHarness**）定义为一个可编程层，它封装现有的静态环境，并将其转化为可定制环境（定制环境 = 静态环境 + 环境挂载器）。环境挂载器通过修改流经标准接口的信息来实现这种定制，同时完全不动底层环境。与智能体挂载器的工具和记忆类似，环境挂载器由模块化插件组件组装而成，这些组件根据特定训练需求定制环境，例如隔离特定技能、延长任务时间范围或调整任务难度。

环境挂载器的形式化定义。 本文将环境建模为元组 $E = (S, \mathcal{A}, O, T, R, s_0)$ ，其中 S 为状态空间， $\mathcal{A}$ 为动作空间， O 为观测空间， $T: S \times \mathcal{A} \rightarrow S$ 为将状态和动作映射到下一状态的转移函数， R 为由验证器产生的奖励， s_0 为初始状态。环境挂载器组件是一种与环境无关的变换 w ：

$$E' = w(E), \quad E' = (S', \mathcal{A}', O', T', R', s'_0). \quad (1)$$

w 严格在接口层面重塑环境，而不修改其底层模拟器后端或实现细节。它定制初始状态（ s'_0 ）、过滤暴露的空间（ $\mathcal{A}', O'$ ）并更新转移机制（ T' ）。由于所有干预都保持在外部，真实评估逻辑得以保留，确保原始验证器仍能为回合评分。

2.2. 三种环境挂载器组件

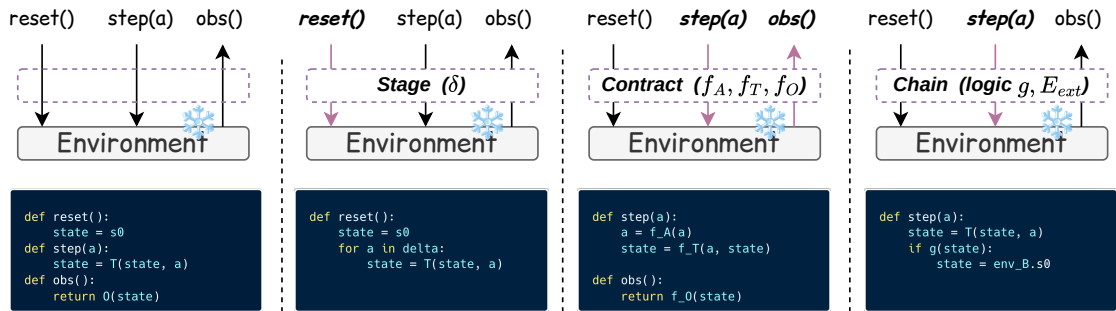


图 3 | 环境挂载器组件封装标准环境接口的概览。底层基础环境（原生状态转移和原始任务验证器）保持完全冻结。从左到右：基础环境，随后是三种环境挂载器组件——阶段（Stage）、契约（Contract）和链（Chain）。高亮箭头和标题指示被覆盖的接口方法，代码块展示每个封装器如何在不改变基础环境的情况下修改状态初始化、转移动态或观测处理。

映射 w of Eq. (1) 定义了一个通用接口，任何遵循该接口的变换都是有效的环境驾驭组件。本文引入了三种具体组件类型，旨在覆盖环境定制三种基本模式，并预期未来会有更多类型。每种类型由其自身参数指定，并重写标准环境接口方法，如重置或步进。图 3 对比了三种组件类型与原始接口，突出显示了每种组件所重写的具体方法。本文在附录 C 中详细说明了该接口协议和类架构的软件实现。在本小节中，本文以 ALFWorld 任务“将干净的马克杯放在桌子上”为例进行说明，该任务的默认实例将马克杯置于开放环境中，并在放置后立即结束。

阶段：改变初始状态。阶段 $\cdot w_{\text{stage}, \delta}$ 由一系列状态操作动作 $\delta = (a_1, \dots, a_k)$ 指定。这些动作应用于由重置函数产生的初始状态 s_0 ：

$$E' = w_{\text{stage}, \delta}(E) = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, R, s'_0), \quad \text{where } s'_0 = T(\dots T(T(s_0, a_1), a_2) \dots, a_k). \quad (2)$$

在此变换下，仅初始状态发生变化。阶段（Stage）定制智能体的起始点，要么引入需要特定技能才能应对的障碍，要么提前完成早期子目标以缩短任务时间跨度。例如，在本文的运行任务中，一个阶段在重置（reset）返回的状态上执行以下动作序列：拿起杯子 1，打开抽屉 1，将杯子 1 放入抽屉 1，并关闭抽屉 1。该阶段有意将杯子隐藏起来，迫使智能体面对更具挑战性的场景：先搜索物体，而不是直接伸手去拿眼前可见的物体。相反，另一个阶段可以通过提前执行清洗杯子的动作来简化设置，只留下最后的放置步骤。

合同：重写交互过程。契约 $\cdot w_{\text{contract}, r}$ 由三个变换映射 $r = (f_A, f_r, f_O)$ 组成，每个映射默认均为恒等映射。这些映射分别重写动作空间、转移动态和观测空间：

$$E' = w_{\text{contract}, r}(E) = (\mathcal{S}, \mathcal{A}', \mathcal{O}', T', R, s_0), \quad \text{where } (\mathcal{A}', \mathcal{O}', T') = (f_A(\mathcal{A}), f_O(\mathcal{O}), f_T(T)). \quad (3)$$

在实践中，这些变换强制执行动作前置条件、增强或屏蔽观测，并将结构化反馈附加到特定结果上以引导智能体学习。例如，在我们的运行任务中，一个契约配置 f_O 以截断房间描述的前两句，要求智能体在多个步骤中构建其空间表示；另一个契约使用 f_r 在智能体未持有杯子时阻止清洁杯子的动作，迫使智能体先拾取该物体；第三个契约配置 f_A 以移除高级传送导航命令，迫使智能体逐步移动和搜索。

链：扩展环境。链 $\cdot w_{\text{chain}, \ell}$ 由一对 $\ell = (E_{\text{ext}}, g)$ 指定，其中 E_{ext} 是一个附加环境， g 是一个组合逻辑。组合逻辑将原始环境 E 和 E_{ext} 组合成一个通过相同接口暴露的复合环境 E' ：

$$E' = w_{\text{chain}, \ell}(E) = (\mathcal{S}', \mathcal{A}', \mathcal{O}', T', R', s'_0), \quad \text{where } E' = g(E, E_{\text{ext}}). \quad (4)$$

为了实现跨环境组合，新空间只是基础环境的并集（例如， $\mathcal{A}' = \mathcal{A} \cup \mathcal{A}^{\text{ext}}$ ），并且 R' 充当新的复合奖励。组合逻辑 g 不受限制，允许环境根据中间结果动态地串联、交错或分支（示例见附录 D）。组合逻辑 g 可以从一开始就组合两个环境，或者使用转移函数 T' 在满足特定条件时动态地从一个环境过渡到下一个环境。对于我们的运行环境，一个链在同一房屋中追加“加热一个土豆并将其放在台面上”，仅在两个环境都得到验证时在 R' 下返回成功。这要求智能体学会在原本会停止的点之后继续携带其目标。

组合。由于所有环境框架组件共享标准接口，它们可以自由组合。例如，在我们的基础杯子任务上堆叠一个阶段、一个契约和一个链，产生一个单一的复合环境：

$$E' = w_{\text{chain}, \ell} \left(w_{\text{contract}, r} \left(w_{\text{stage}, \delta}(E) \right) \right). \quad (5)$$

在此设置中， w_{trap} 初始化回合时将杯子隐藏在抽屉中以强制空间搜索； w_{contract} 将观测截断为两句话以评估部分可观测性； w_{chain} 追加一个后续任务以测试目标持久性。注意，这些变换是不可交换的（ $w_1 \circ w_2 \neq w_2 \circ w_1$ ）；嵌套顺序决定了结果环境的构建方式以及在初始化与主动交互期间应用哪些约束。

3. 用于智能体学习的环境框架

3.1. 问题设定

给定一个基础环境 E ，它支持一组基础任务，以及一个目标策略智能体 π ，我们的目标是自动生成一个针对特定任务 t 定制的修改环境 E' ，暴露 π 的独特缺陷，以促进有针对性的策略改进。单个环境测试框架（EnvHarness）组件本身是与策略无关的，因为公式（1）将 w 定义为仅对环境的变换，允许同一组件无需修改即可应用于任何策略。相反，这些组件的选择和参数化必须同时以基础任务 t 和策略 π 的观察行为为条件。因此，我们引入任务-策略条件映射 H ：

$$E' = \mathcal{H}(E, t; \pi) = (w_k \circ w_{k-1} \circ \dots \circ w_1)(E), \quad (6)$$

其中每个 w_i 是一个定制的环境封装组件，旨在包装基础环境 E ，并暴露 π 在任务 t 上的关键弱点。这些组件不检查内部模型权重，而是将智能体视为黑盒，仅基于其输出生成稳定、纠正性的训练信号。

3.2 环境装配器

本文引入环境装配器（EnvRigger）以实现式（6）中任务-策略条件映射 H 。环境装配器在任务 t 上于环境中运行 π ，分析所得轨迹，编写特定的环境装配组件（EnvHarness）以针对该任务定制环境，并利用全新策略回放验证定制后的环境。来自提供适当学习信号的定制环境的候选方案被接受，而失败候选方案的环境装配组件则被拒绝或修订。图 4 展示了这一完整工作流程，其中环境装配器系统性地经历四个不同阶段：观察、诊断、编写与验证。为确保阶段引入的初始状态变异能够可靠复现，本文假设基础环境在验证阶段支持确定性重置。

观察。环境装配器（EnvRigger）首先在当前环境中对基础任务 t 运行策略 π ，以收集并分析一批展开轨迹。失败暴露了该任务中需要解决的具体弱点，而成功则有助于界定这些缺陷的边界，表明哪些能力已经完好，以及它们在何处开始失效。

诊断。环境装配器（EnvRigger）分析收集到的轨迹，以识别所观察行为的根本原因，重点关注系统性问题，如重复动作循环、解析长观测失败或误读工具约束。该诊断还确定了定制方向。对于表现不佳的策略，目标是搭建缺失步骤并简化任务。相反，如果策略达到完美成功率，则表明当前环境过于宽松，未能暴露

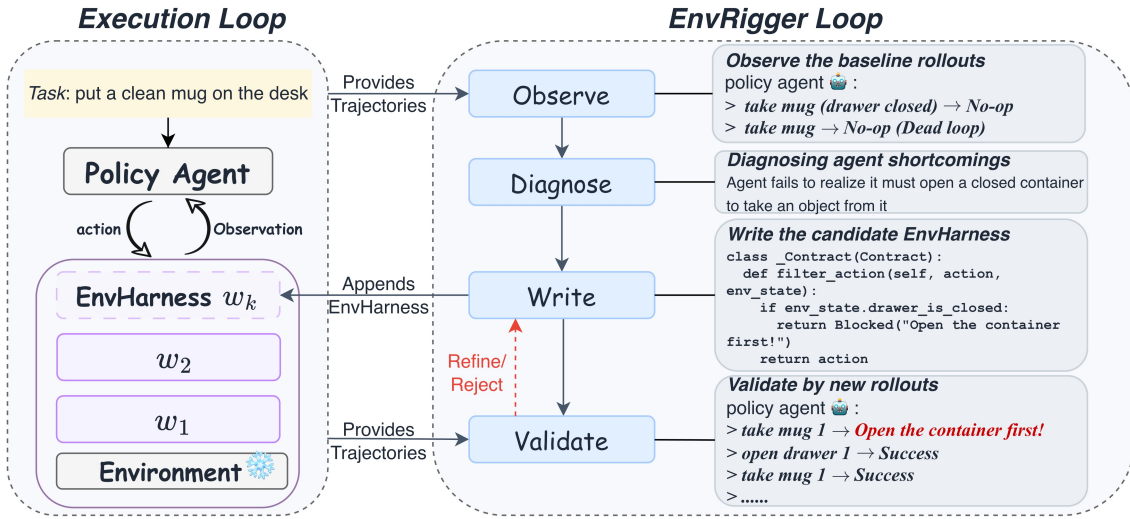


图 4 | 环境装配器 (EnvRigger) 根据给定任务为目标策略生成环境装配组件 (EnvHarness components)。左侧的执行循环在当前环境中运行策略，该环境是由包含已接受组件 $w_1, \dots, w_k$ 的活动环境装配 (EnvHarness) 包裹的冻结基础环境，而生成的轨迹结果输入右侧的环境装配器 (EnvRigger) 循环。环境装配器 (EnvRigger) 系统地通过四个不同阶段运行：观察、诊断、编写和验证，其中后两个步骤形成一个编写-验证循环，生成候选组件，在新轨迹上评估它，并在失败时进行修订。

任何剩余的弱点。在这种情况下，环境装配器 (EnvRigger) 诊断出必须使环境更难，将定制转向注入更具挑战性的场景，迫使策略的潜在缺陷暴露出来。环境装配器 (EnvRigger) 随后将这些发现输出为文本诊断。

编写。基于诊断，环境装配器 (EnvRigger) 合成一个或多个环境装配组件 (EnvHarness components) 以针对已识别的缺陷。单个缺陷可能需要组合多个组件，例如一个定制初始状态的阶段 (Stage) 和一个过滤后续交互的契约 (Contract)，它们作为候选集一起发出。例如，如果诊断显示策略依赖于绕过学习的脆弱捷径，环境装配器 (EnvRigger) 可以编写一个契约 (Contract)，在特定条件下阻止该动作，迫使策略探索并掌握预期技能。

验证。为了评估编写阶段产生的候选组件，环境装配器 (EnvRigger) 利用这些组件包装当前环境以实例化 E' ，并在基础任务 t 上对 π 进行全新的轨迹采样。基于某些轨迹度量，例如成功率和失败分布，环境装配器分析这些全新采样轨迹，以在三种验证行为之间做出决策：接受候选组件、拒绝那些不可解或缺乏挑战性的候选组件，或对信号缩放不佳的候选组件进行细化。如果候选组件需要细化，轨迹和缩放反馈将回流至编写阶段，重复此编写-验证循环，直至某个候选组件被接受或修订预算耗尽。所有被接受的组件最终被添加至环境测试装置 (EnvHarness)。指导这些验证行为的精确系统提示和决策标准详见附录 A。

4. 实验

4.1. 实验设置

本节主要关注基于技能的学习范式，其中技能从环境中提取以提升智能体能力。同时，我们评估了本框架在在线强化学习范式下的兼容性与性能，该部分作为更广泛分析的一部分呈现于第 5 节。

基准与评估。我们在涵盖四个不同领域的五个基准上评估本框架：用于文本具身环境的 ALFWorld (Shridhar 等, 2020)、用于网页交互的 WebArena (Zhou 等, 2024)、用于软件工程的 SWE-bench Verified (Jimenez 等, 2024)，以及用于办公自动化的 OfficeQA (Databricks, 2025) 与 SpreadsheetBench (Ma 等, 2024)。我们报告每个基准的原生度量，并额外跟踪 SWE-bench Verified 上的平均步数作为执行效率的测度。训练与评估回合在每个基准上严格分离，详细划分配置见附录 E.1。

重要的是，环境装配器 (EnvRigger) 与策略智能体在每个基准上使用相同的模型骨干：在 ALFWorld 和 WebArena 上使用 Gemini-3.1-Flash-Lite，在其他基准上使用 Gemini-3.5-Flash，确保性能提升并非源于蒸馏更强的外部模型。在每个训练集上，环境装配器执行第 3 节的优化循环，生成环境测试台 (EnvHarness) 定制的环境。随后，我们按照推理银行 (ReasoningBank) (欧阳等人, 2025) 从这些环境中收集的轨迹中提取技能，并在留出实例上评估配备技能的智能体。由于环境装配器难以观察连接环境的内部状态，我们将链式组件排除在此自动化流程之外，并在第 5 节单独分析链式效果。

基线方法。我们将环境测试台与四种技能来源进行比较：无技能（仅冻结策略智能体）、原始环境（从原始环境提取技能以隔离重塑效果），以及各自基准上的生成环境 (**GenEnv**) (郭等人, 2025)、验证环境 (**VeriEnv**) (蔡等人, 2026) 或软件工程锻造 (**SWE-smith**) (杨等人, 2026a)。这些基线生成器是领域特定的，而环境测试台通过统一接口在所有领域通用，包括没有生成基线的办公自动化环境。为确保公平比较，所有基线共享相同的种子实例、环境数量、提取流程和策略模型。环境装配器严格在训练回合上运行，并具有相同的预言机验证访问权限，每个评估实例仅尝试一次。基线详情见附录 E.2。

4.2. 主要结果

表 2 和表 3 总结了五个基准的主要结果。基于这些评估，我们提出以下关键观察。

环境测试台在静态环境无法提供增益的地方带来一致提升。在环境测试台定制的环境中获得的技能在每个基准上都始终优于从原始环境提取的技能，在 ALFWorld 上提升高达 9.0 分。相反，从静态基础环境提取技能实际上可能降低性能；例如，在电子表格基准 (SpreadsheetBench) 上，来自未修改环境的技能低于无技能基线，而在软件工程基准验证集 (SWE-bench Verified) 上，它们延长了执行轨迹。由于静态环境仅允许

表 2 | 在 ALFWorld 和 WebArena 上，配备从不同环境来源提取技能的智能体的性能。所有数值均为三次独立运行的平均值，标准差以灰色下标表示。所有度量指标均为越高越好，最后一行报告了 EnvHarness 环境相对于原始环境的改进。“-”表示该方法仅适用于特定基准，无法应用于其他领域。

Skill Source	ALFWorld			WebArena				
	In-Dist	OOD	Avg.	Reddit	Shopping	Shop Admin	GitLab	Avg.
No Skills	62.6 _{1.7}	60.7 _{5.2}	61.7 _{3.4}	39.6 _{2.3}	35.2 _{3.3}	44.1 _{2.3}	35.8 _{8.4}	38.7 _{2.3}
Original Envs	63.3 _{2.8}	61.4 _{4.3}	62.4 _{3.4}	38.7 _{9.7}	35.2 _{1.3}	44.6 _{3.0}	35.4 _{4.0}	38.5 _{3.1}
GenEnv	63.3 _{1.2}	61.9 _{2.7}	62.6 _{1.9}	-	-	-	-	-
VeriEnv	-	-	-	39.6 _{4.2}	30.2 _{0.0}	49.7 _{2.4}	38.9 _{5.6}	39.6 _{1.4}
ENVHARNESS Envs	66.2 _{0.3}	70.4 _{2.3}	68.3 _{1.3}	40.6 _{4.7}	37.4 _{0.3}	50.8 _{1.5}	37.7 _{3.1}	41.6 _{1.8}
Improvement	+2.9	+9.0	+5.9	+1.9	+2.2	+6.2	+2.3	+3.1

表 3 | 在 SWE-bench Verified、OfficeQA 和 SpreadsheetBench 上，配备从不同环境来源提取技能的智能体的性能。标准差以灰色下标表示。“-”表示该方法仅适用于特定基准，无法应用于其他领域。

Skill Source	SWE-verified		OfficeQA		SpreadsheetBench	
	SR (↑)	Average Step (↓)	EM (↑)	F1 (↑)	Pass@1 (↑)	Mean Score (↑)
No Skills	47.67 _{0.93}	53.58 _{2.93}	54.23 _{2.84}	55.77 _{2.98}	46.44 _{0.15}	61.32 _{0.37}
Original Envs	49.88 _{2.59}	55.01 _{1.69}	54.40 _{1.84}	55.77 _{1.59}	45.88 _{1.19}	61.47 _{0.59}
SWE-smith	50.12 _{1.74}	54.72 _{2.03}	-	-	-	-
ENVHARNESS Envs	52.58 _{2.72}	49.61 _{2.49}	56.20 _{2.34}	57.73 _{2.29}	49.15 _{0.36}	62.48 _{0.27}
Improvement	+2.70	+5.40	+1.80	+1.96	+3.27	+1.01

智能体练习其已执行的行为，但未能解决其特定局限性，往往检索到冗余或次优的技能。相比之下，EnvRigger 的写入-验证循环仅提交经新策略轨迹验证的环境组件，确保 EnvHarness 在所有基准上始终优于无技能基线。

EnvHarness 通过领域无关的接口实现跨领域泛化。底层接口协议、EnvRigger 循环和技能提取流程在所有五个基准上一致应用，仅需领域特定的提示模板以适应不同环境。相比之下，专门的生成基线受限于其特定目标基准（在表 2 和表 3 中不适用领域以破折号表示）。然而，EnvHarness 在可应用的领域始终优于这些专门基线。在 ALFWorld 上，EnvHarness 技能平均比 GenEnv 高出 5.7 分，在分布外设置中高出 8.5 分，而通用实例生成仅增加重复练习，未能解决策略弱点。在 SWE-bench Verified 上，EnvHarness 的成功率比专用 SWE-smith 高出 2.46 分，同时每个回合少用 5.11 个执行步骤。通过统一接口针对诊断出的漏洞进行改进，因此优于通过领域特定生成单纯扩大训练回合数量。

EnvHarness 通过修复浪费行为来提高效率。在 SWE-bench Verified 上，从 EnvHarness 定制环境中提取的技能将每个回合的平均步骤数从 53.6 降至 49.6，而从未修改环境中提取的技能反而将其增加到 55.0。这

效率提升与 EnvRigger 的具体诊断直接相关：旨在打破重复动作循环并过滤冗长观测的目标契约与阶段，成功缩短了执行轨迹。

5. 分析

本文从五个维度分析 EnvHarness：其作为强化学习训练信号的兼容性、相较于标准数据集扩展的扩展效率、在不同策略模型家族与能力水平间的可迁移性、链组件在长时程环境中的独特价值，以及 EnvRigger 接受显式用户定义目标约束的能力。更多分析见附录 G。

EnvHarness 实现更优的强化学习。除基于技能的学习外，本文还探究 EnvHarness 定制环境能否在在线强化学习中提供主动训练信号。本文在 ALFWorld 与 WebShop (Yao 等, 2022a) 上进行分析，以 Qwen3-8B-base (Yang 等, 2025) 作为策略，通过分组相对策略优化 (GRPO) (Shao 等, 2024) 进行优化；训练细节见附录 F.1。本文训练两个独立策略：一个仅在原始静态环境上训练，另一个完全在由 EnvHarness 重塑的环境上训练，并在相同的留出实例上评估两者。表 4 报告了结果。在 EnvHarness 环境上训练持续提升策略性能，在四项度量中的三项上优于仅在原始环境上训练的基线。具体而言，在 ALFWorld 上，EnvHarness 环境达到 87.9 的分布内成功率，而原始环境为 81.4。在 WebShop 上，其获得更高的得分 79.2 (对比 75.6) 与成功率 67.4 (对比 66.0)。尽管 ALFWorld 分布外成功率存在轻微且可忽略的权衡 (88.8 对比 89.6)，但总体结果凸显了一个根本优势：重塑的环境不仅是辅助数据，更为在线策略学习提供了高效且独立的优化信号。

Training Set	ALFWorld			WebShop	
	In-Dist	OOD	Avg. Score	SR	
Original Envs	81.4	89.6	85.5	75.6	66.0
ENVHARNESS Envs	87.9	88.8	88.4	79.2	67.4

表 4 | 在 ALFWorld 与 WebShop 上的强化学习，比较在原始环境与 EnvHarness 环境上训练的策略。ALFWorld 以分布内与留出实例类型的成功率评分；WebShop 报告环境得分与成功率。

链式机制 (Chain) 能够高效地解决长时域任务。现实世界的应用通常要求智能体在较长的时域内持续运行。链式组件通过将两个随机配对的基础环境合并为一个单一的、延长的回合来解决这一问题。为了隔离其效果，这种配对独立于自主环境装配器 (EnvRigger) 循环运行。我们在标准的单环境测试实例上评估提取的技能，并在表 5 中报告成功率 (SR) 和平均步数 (AS)。

	SR (%) ↑	AS ↓
No Skills	47.67	53.58
Original Envs	49.88	55.01
ENVHARNESS (Stage/Contract Only)	52.58	49.61
ENVHARNESS (Chain Only)	49.63	41.96
Combined Skills (Stage/Contract + Chain)	54.30	43.12

表 5 | 长时域环境上的性能。SR 表示成功率，AS 表示平均步数。

来自链式环境的技能带来了显著的效率提升，将 AS 从 53.58 降低到 41.96。其独立的 SR (49.63) 略低于 49.88 的基线。这与它们严格的训练条件一致——成功需要同时解决两个部分——这优先考虑长期目标的保持，而非短期任务的最大化。结合两种技能集（阶段 / 契约 + 链式）实现了两全其美：最高的 SR (54.30) 和出色的效率 (43.12 AS)，展示了高度互补的行为。代表性技能见附录 F.3。

环境测试平台 (EnvHarness) 能够高效地构建环境扩展性。

环境扩展性评估在可用训练环境增多时的性能表现，比较了在相同预算下的三种分配策略：EnvHarness 环境、未修改的基准环境以及 SWE-smith 生成的环境。在固定策略模型、环境预算和技能检索协议的情况下，每批 50 个环境生成一个技能库（每个技能库交替包含 2 到 3 个技能，在 300 个环境时总计 15 个技能）。关键的是，两个基线独立于学习者抽取环境批次，而 EnvHarness 则针对配备了先前累积技能的策略专门合成每个批次，使环境和策略能够共同进化。如图 5 所示，EnvHarness 从 47.67 提升至 54.79（提高了 7.12 分），并在 300 个环境时保持上升趋势。相比之下，相同的预算在原始环境上仅达到 52.13，在生成环境上仅达到 50.37。这一性能差距证实，针对学习者当前能力边界进行定向比无条件的环境扩展从根本上更有效。每轮的代表性技能见附录 F.2。

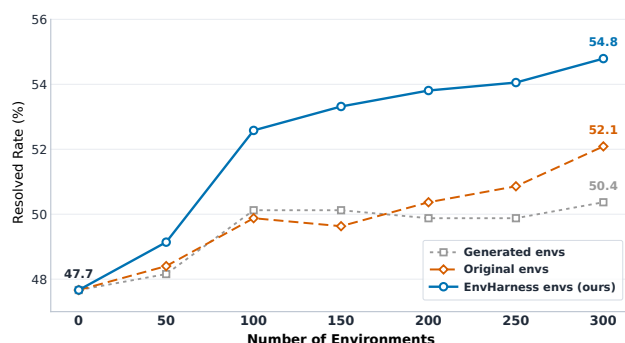


图 5 | 在 SWE-bench Verified 上的环境扩展。所有三个来源提供相同数量的环境，并采用相同的提取和检索协议。

EnvHarness 在不同大语言模型骨干上具有泛化能力。

为评估泛化性，我们在 SWE-bench Verified 上测试了四种不同的模型：Gemini 3.1 Flash-Lite、Qwen3.6 27B (Qwen Team, 2026)、Gemini 3.5 Flash 和 Claude Sonnet 4.6 (Anthropic, 2026a)。这些模型涵盖了开放权重和专有架构，跨越了广泛的能力范围。在每种设置中，我们对目标策略和 EnvRigger 使用相同的模型骨干以保持设置一致，而提取流程和协议保持不变。

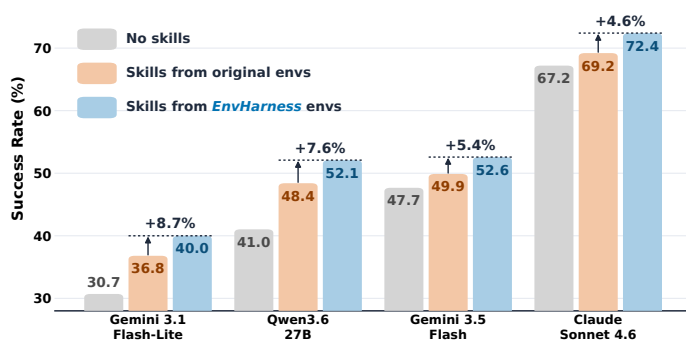


图 6 | 在 SWE-bench Verified 上的跨模型结果。每组代表一个策略模型，按从弱到强排序。柱状图显示无技能、使用未修改环境技能以及使用 EnvHarness 环境技能时的成功率。百分比表示相对于原始环境的相对提升。

图 6 展示了结果。EnvHarness 技能在所有四个策略上均优于真实环境技能，绝对提升幅度为 2.7 到 3.7 分，尽管无技能成功率跨度很大，从 30.7 到 67.2。这一提升幅度在很大程度上与底层策略的强弱无关：定制化循环既不会在最弱的模型上失效，也不会最强模型上饱和，并且相同的流程，

提示词和验收标准贯穿始终。策略能力水平的变化似乎改变了诊断的内容，而非循环的适用性。我们注意到，相对于完全没有技能的情况，两类技能对两个最弱策略的帮助最大（环境测试框架提升 9.3 和 11.1 分，未修改环境提升 6.1 和 7.4 分，而两个最强策略的提升不足 5.5 分）。

环境测试框架按需生成环境。虽然环境装配循环在标准设置中通过行为诊断自主识别训练目标，但同样的机制可以轻松接受显式的、用户定义的约束。我们评估两类约束。第一类是附录 G 中客观性能度量的定量目标（成功率或平均步数）。第二类约束针对用自然语言描述的能力弱点。例如，给定以下弱点，环境装配器生成一个契约，除非运行测试，否则拒绝代码提交。这迫使智能体验证其修复。从生成的轨迹中，我们提炼出如下所示的技能。该技能并非对单一任务过拟合，而是将一般原则与可操作步骤相结合，完全符合我们对技能的要求（Yang 等人，2026b）。

Specified weakness

The policy submits a patch without running the failing test, so the fix stays unverified.

Generated component (Contract, f_T axis)

```
class _Contract(Contract):
    def modify_transition(self, action, response, env_state):
        cmd = bash_command(action)
        if "pytest" in cmd or "runtests.py" in cmd:
            env_state.extras["ran_tests"] = True
        if is_submission(cmd) \
            and not env_state.extras.get("ran_tests"):
            return failed(response,
                "githook: pre-commit hook 'verify-tests' "
                "failed. Run the test suite before submitting.")
        return response
```

Verification-Driven Development Loop

描述：每当为修复错误或实现功能而更改代码时，尤其是测试套件需要设置或配置时。内容：在最终确定任何更改之前，运行相关测试套件以确认故障存在，然后在补丁后再次运行以验证修复，必要时先初始化环境。

6. 相关工作

6.1. 环境扩展

环境扩展已成为一个研究方向，为智能体提供更多可供学习的环境（Huang 等人，2025b；Xi 等人，2025）。现有工作以各种形式扩展环境，包括使用大语言模型模拟环境及其反馈（Guo 等人，2025；Wang 等人，2025；Zala 等人，2024），直至模拟或合成整个环境族的世界模型

智能体环境 (Agentic Environments) (Wang 等, 2026; Zuo 等, 2026), 以编程方式合成可执行环境 (Chae 等, 2026; Dong 等, 2026; Song 等, 2026; Sun 等, 2026; Tang 等, 2026), 以及在现有基准 (Benchmark) 内合成新任务实例 (Pan 等, 2024; Yang 等, 2026a)。除了生成更多任务之外, 另一条研究路线调整环境呈现给学习者的内容, 从强化学习 (Reinforcement Learning) 中的课程生成 (Dennis 等, 2020; Jiang 等, 2021; Liu 等, 2026; Wang 等, 2019) 到手工设计的纠正性反馈和奖励塑形 (Reward Shaping) (Lu 等, 2025)。与以往依赖特定基准流程或手工设计课程的工作不同, EnvHarness 通过一个跨基准共享的接口重塑现有环境, 根据当前策略诊断出的弱点来调整重塑过程, 并且不改变任务和验证器 (Verifier)。

6.2. 自我进化智能体

自我进化智能体 (Self-Evolving Agents) 在没有额外人类监督的情况下, 从自身经验中改进自己 (Fang 等, 2025)。现有工作进化智能体的不同部分, 包括提示和反思 (Reflection) (Madaan 等, 2023; Shinn 等, 2023), 技能和工作流库 (Huang 等, 2026b; Wang 等, 2023, 2024; Xia 等, 2026a, b; Yang 等, 2026b), 从过去轨迹中提炼的经验记忆 (Ouyang 等, 2025; Zhao 等, 2024), 通过自生成奖励或自提出任务来更新模型权重 (He 等, 2025; Huang 等, 2025a, 2026a; Xia 等, 2025; Yuan 等, 2024; Zhao 等, 2026), 以及最近围绕冻结模型重写和测试的智能体框架本身 (Lee 等, 2026b)。与这些在固定学习世界中进化智能体的方法不同, EnvHarness 针对冻结策略诊断出的弱点来重塑环境本身。

7. 结论

我们引入了环境驾驭器 (EnvHarness), 这是一个可编程层, 能够将静态的现有环境转变为可控环境。环境驾驭器通过三个插件组件——阶段 (Stage)、契约 (Contract) 和链 (Chain)——封装冻结的基准, 并通过标准的重置 / 步骤接口对其进行全面重塑, 从而能够在从未为此类目的构建的环境中隔离技能、扩展任务视野或校准难度。由于环境驾驭器从不触及内部代码, 单一实现即可在不同领域间无缝工作。此外, 通过保持原始任务不变, 每个重塑后的环境都能安全地保留其可信的、由人类构建的验证器。为了完全自动化这种定制, 我们引入了环境装配器 (EnvRigger), 这是一个自主循环, 它从执行轨迹中诊断策略弱点, 并合成针对性的环境驾驭器组件以提供精确的学习信号。这将环境构建重新定义为封装问题而非创作问题, 并为智能体学习提供了一条通往可扩展环境供应的实用路径。我们在附录 I 和附录 H 中提出了未来方向和局限性。

参考文献

- Anthropic. Effective harnesses for long-running agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, 2025. Accessed: 2026-07-07.
- Anthropic. Claude 4.6 sonnet. <https://www.anthropic.com/news/claude-sonnet-4-6>, 2026a. Accessed: 2026-07-28.

- Anthropic. Harness design for long-running application development. <https://www.anthropic.com/engineering/harness-design-long-running-apps>, 2026b. Published: March 24, 2026. Accessed: 2026-07-07.
- M. Beukman, S. Coward, M. Matthews, M. Fellows, M. Jiang, M. Dennis, and J. Foerster. Refining minimax regret for unsupervised environment design. *arXiv preprint arXiv:2402.12284*, 2024.
- H. Chae, J. Park, and A. Ritter. Safe and scalable web agent learning via recreated websites. *arXiv preprint arXiv:2603.10505*, 2026.
- Databricks. Officeqa: A grounded reasoning benchmark, 2025.
- M. Dennis, N. Jaques, E. Vinitzky, A. Bayen, S. Russell, A. Critch, and S. Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.
- G. Dong, J. Lu, J. Huang, W. Zhong, L. Liu, S. Huang, Z. Li, Y. Zhao, X. Song, X. Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.
- J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, et al. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025.
- J. Guo, L. Yang, P. Chen, Q. Xiao, Y. Wang, X. Juan, J. Qiu, K. Shen, and M. Wang. Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators. *arXiv preprint arXiv:2512.19682*, 2025.
- I. Gur, H. Furuta, A. Huang, M. Safdari, Y. Matsuo, D. Eck, and A. Faust. A real-world webagent with planning, long context understanding, and program synthesis. In *International Conference on Learning Representations*, volume 2024, pages 52690–52717, 2024.
- Y. He, C. Huang, Z. Li, J. Huang, and Y. Yang. Visplay: Self-evolving vision-language models from images. *arXiv preprint arXiv:2511.15661*, 2025.
- Y. Hu, Z. Wen, X. Liu, P. Wang, X. Zhang, and W. Wu. Seal: Synergistic co-evolution of agents and learning environments. *arXiv preprint arXiv:2605.24426*, 2026.
- C. Huang, W. Yu, X. Wang, H. Zhang, Z. Li, R. Li, J. Huang, H. Mi, and D. Yu. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*, 2025a.
- C. Huang, H. Liu, T. Zheng, R. Dai, L. Huang, J. Li, Z. Li, Z. Wei, Y. Meng, and J. Huang. G-zero: Self-play for open-ended generation from zero data. *arXiv preprint arXiv:2605.09959*, 2026a.
- Y. Huang, S. Li, M. Liu, W. Liu, S. Huang, Z. Fan, H. P. Chan, and Y. R. Fung. Environment scaling for interactive agentic experience collection: A survey. *arXiv preprint arXiv:2511.09586*, 2025b.
- Z. Huang, J. Xu, Y. Yang, Z. Gong, Q. Yang, M. Tian, X. Wang, C. Lv, X. Gao, Q. Dai, et al. From raw experience to skill consumption: A systematic study of model-generated agent skills. *arXiv preprint arXiv:2605.23899*, 2026b.
- M. Jiang, E. Grefenstette, and T. Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pages 4940–4950. PMLR, 2021.

- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- S. Lee, S. Chowdhury, C. Jiang, C.-Y. Hsieh, T.-Y. Hu, A. T. Toshev, O. Tuzel, and R. Vemulapalli. Environment-free synthetic data generation for api-calling agents. *arXiv preprint arXiv:2607.16900*, 2026a.
- Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026b.
- J. Li, Z. Jin, T. Men, Y. Hao, K. Zhu, L. Wang, D. Huang, L. Wang, S. Hua, L. Wang, et al. Agentic environment engineering for large language models: A survey of environment modeling, synthesis, evaluation, and application. *arXiv preprint arXiv:2606.12191*, 2026.
- B. Liu, S. Yu, Y. Jiang, A. Qu, A. Zhao, Z. Liu, J. Kim, Z. Zhou, S. Kim, T. Ren, M. Liu, H. Yu, Z. Chen, W. Shi, P. P. Liang, L. Zettlemoyer, Y. Choi, and N. Jaques. Spade: Self-play in adaptive synthetic executable environments. *arXiv preprint arXiv:2608.19197*, 2026. URL <https://arxiv.org/abs/2608.19197>.
- S. Lu, Z. Wang, H. Zhang, Q. Wu, L. Gan, C. Zhuang, J. Gu, and T. Lin. Don’t just fine-tune the agent, tune the environment. *arXiv preprint arXiv:2510.10197*, 2025.
- Z. Ma, B. Zhang, J. Zhang, J. Yu, X. Zhang, X. Zhang, S. Luo, X. Wang, and J. Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. *arXiv preprint arXiv:2406.14991*, 2024.
- A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- Q. Meng, Y. Wang, L. Chen, W. Wu, Y. Li, W. Jiang, Q. Wang, C. Lu, Y. Gao, Y. Wu, and Y. Hu. Agent harness for large language model agents: A survey. 2026. doi: 10.20944/preprints202604.0428.v3. URL <https://www.preprints.org/manuscript/202604.0428/v3>.
- M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- OpenAI. Harness engineering: leveraging codex in an agent-first world. <https://openai.com/index/harness-engineering/>, February 2026. Accessed: 2026-07-07.
- S. Ouyang, J. Yan, I. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. T. Le, S. Daruki, X. Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026. URL <https://qwen.ai/blog?id=qwen3.6-27b>.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.
- X. Song, H. Chang, G. Dong, Y. Zhu, J.-R. Wen, and Z. Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 8326–8357, 2026.
- S. Sun, H. Song, L. Huang, J. Jiang, R. Le, Z. Lv, Z. Chen, Y. Hu, W. Luo, W. X. Zhao, et al. Swe-world: Building software engineering agents in docker-free environments. *arXiv preprint arXiv:2602.03419*, 2026.
- Z. Tang, Y. Liu, X. Lai, J. Li, P. Lyu, Y. Guo, Z. Fang, Y. Ding, Y. Zhang, W. Wang, et al. Phoneworld: Scaling phone-use agent environments. *arXiv preprint arXiv:2605.29486*, 2026.
- B. Trabucco, G. Sigurdsson, R. Piramuthu, and R. Salakhutdinov. Insta: Towards internet-scale training for agents. *arXiv preprint arXiv:2502.06776*, 2025.
- G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- R. Wang, J. Lehman, J. Clune, and K. O. Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions. *arXiv preprint arXiv:1901.01753*, 2019.
- Y. Wang, D. Yin, Y. Cui, R. Zheng, Z. Li, Z. Lin, D. Wu, X. Wu, C. Ye, Y. Zhou, et al. Llms as scalable, general-purpose simulators for evolving digital agent training. *arXiv preprint arXiv:2510.14969*, 2025.
- Z. Wang, C. Xu, B. Liu, Y. Wang, S. Han, Z. Yao, H. Yao, and Y. He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026.
- Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- Z. Xi, Y. Ding, W. Chen, B. Hong, H. Guo, J. Wang, X. Guo, D. Yang, C. Liao, W. He, et al. Agentgym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27914–27961, 2025.
- P. Xia, K. Zeng, J. Liu, C. Qin, F. Wu, Y. Zhou, C. Xiong, and H. Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025.
- P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026a.
- P. Xia, J. Chen, X. Yang, H. Tu, J. Liu, K. Xiong, S. Han, S. Qiu, H. Ji, Y. Zhou, et al. Metaclaw: Just talk—an agent that meta-learns and evolves in the wild. *arXiv preprint arXiv:2603.17187*, 2026b.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- J. Yang, K. Lieret, C. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents. *Advances in Neural Information Processing Systems*, 38, 2026a.
- Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, et al. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026b.
- S. Yao, H. Chen, J. Yang, and K. Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022a.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022b.
- W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.
- A. Zala, J. Cho, H. Lin, J. Yoon, and M. Bansal. Envgen: Generating and adapting environments via llms for training embodied agents. *arXiv preprint arXiv:2403.12014*, 2024.
- L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025.
- A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- A. Zhao, Y. Wu, T. Wu, Q. Xu, Y. Yue, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang. Absolute zero: Reinforced self-play reasoning with zero data. *Advances in Neural Information Processing Systems*, 38:105816–105879, 2026.
- S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.
- Y. Zuo, Z. Xiao, L. Sheng, F. Huang, J. Tu, Y. Liu, T. Tang, X. Hu, Y. Su, Q. Lan, et al. Qwen-agentworld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.

附录内容

A 环境装配器提示词 19

B 与相关协同进化和综合框架的显著差异 20

C 接口协议与设计模式 21

C.1 可操作环境：可交互环境接口 21

C.2 桥接器：适配异构基准 22 C.3 环境驾驭器：作为可组合装饰器的组件 22

D 链（Link）算子的具体实现示例 24

E 实验细节 25 E.1 基准划分 25 E.2 基线细节 25
E.3 EnvRigger 超参数 25

F 分析细节 26 F.1 强化学习实验细节 26 F.2 协同进化轮次中的技能 27
F.3 链式环境中的技能 33 F.4 跨模型结果 33

G 补充分析 35

H 局限性 41

I 未来方向 41

A. EnvRigger 提示词

命名。发布的代码早于本文的术语。三种组件类型在代码中分别以类 `Setups`、`Rules` 和 `Link` 出现，设计器生成的候选对象携带字段 `rules_code` 和 `in_env_actions`。表 6 对两者进行了映射，本附录其余部分使用论文中的名称。

Paper	Code	Emitted field
Stage	Setups	<code>in_env_actions</code>
Contract	Rules	<code>rules_code</code>
Chain	Link	—

表 6 | 论文与发布版本中的组件名称。

以下提示是每个基准共享的部分。每个基准会附加一小段自身内容，详细说明其桥接器暴露的工具、其 `env_state` 视图的字段以及领域特定约束；这些内容块位于代码发布中。

System prompt of the ENVHARNESS designer agent

你是智能体基准的环境设计器。你的任务是重塑环境，使策略智能体获得正确的训练信号。你输出一个具有两个独立杠杆的候选对象：

1. ``rules_code`` -- a Python class ``_Rules(Rules)`` overriding up to three per-step hooks: `filter_action` (A axis: transform or Block an action), `modify_transition` (T axis: transform the env's response), and `filter_observation` (O axis: transform what the Policy sees). All hooks default to pass-through; the class is loaded fresh per episode.
2. ``in_env_actions`` -- a list of tool calls the framework REPLAYS through `env.step()` before the Policy starts. This is the S0 (initial-state) mechanism: instead of writing code, you write a trajectory the environment walks for you.

The two levers compose freely: S0-only, hooks-only, or both (e.g. seed a state via `in_env_actions`, then block the easy escape via `filter_action`). The R axis is not exposed: success is the benchmark's own verdict, so reshaping reward cannot move the eval metric. Hooks may read the `env_state` schema provided each turn; import only the standard library.

PITFALL -- do not make the task unsolvable. A mutation that makes success impossible is not a difficulty increase; SR=0 from impossibility is exactly as useless as SR=1 from triviality. Signals that a prior mutation was unsolvable: most rollouts end in timeout, or SR=0 with failures pointing at the action axis. On these signals your next proposal must REVERSE or loosen the offending restriction -- stacking more bans cannot climb into the band. Prefer subtle, narrow perturbations (one op, one obs key) over sweeping bans.

BASELINE -- before your first proposal you see K unmutated rollouts on this task: success rate, per-rollout outcomes, and sample trajectories. Read it for three things: WHETHER the Policy can solve the task at all (if baseline SR is ~0, "make it harder" is nonsensical -- scaffold it easier or skip); HOW it solves it (a 4-step solution leaves less headroom than a 30-step one -- match perturbation magnitude to that headroom); and WHICH parts of the env it actually relies on (perturbing commands it never uses is irrelevant). Treat the baseline as raw data, not as a hint: decide direction and magnitude yourself.

REFINE -- after K rollouts of your candidate you decide **ACCEPT** / **REFINE** / **REJECT**, referencing rollout statistics (SR over K runs, failure distribution, timeout count), never a single trace. When refining, ask: did this mutation move SR toward the target band? If yes, the perturbation TYPE is right -- keep the working hooks verbatim and adjust only the magnitude (loosen if overshot, tighten if undershot); do not discard code that paid K rollouts of signal to establish. If no, start over with a different perturbation type.

你的运行机制是固定的。每轮提供告诉你优化目标的目标 (**OBJECTIVE**)；每个基准的约束作为实验特定指令附加。

B. 与相关协同进化与综合框架的显著差异

为了在自适应环境设计与协同进化的文献中清晰定位 EnvHarness，我们强调本框架与三个密切相关范式的核心差异：生成式协同进化、自适应配置引擎和程序化环境综合。

1. 与生成式协同进化（例如 GenEnv）的比较。

- **其方法：**GenEnv (Guo 等, 2025) 利用大语言模型作为生成式模拟器，动态生成状态转移、观测和成功信号。
- **其局限性：**依赖大语言模型模拟物理转移并验证任务，本质上会引入幻觉和评估漂移，从而损害基准的数学有效性。
- **我们的解决方案：**EnvHarness 完全冻结基础环境、其原生转移函数 T 以及可信的人工构建验证器。所有定制均通过阶段、契约和链包装器在标准接口边界非侵入式应用，确保 100% 确定性的转移逻辑和高可信度的评估完整性。

2. 与自适应配置引擎（如 EnvGen）的比较。

- **其方法：**EnvGen (Zala 等, 2024) 在模拟器核心内部生成并调整环境配置（例如更改地图或地形文件）。
- **其局限性：**修改模拟器的内部代码或资源配置高度依赖于特定基准，需要针对每个新领域进行深入的人工工程，并可能破坏状态逻辑。
- **我们的解决方案：**EnvHarness 完全不受基准限制。虽然添加新环境需要实现一次性的轻量级包装桥接（标准可操作环境接口），但核心协同进化循环和环境设计器无需进一步修改。一旦接口建立，EnvHarness 会自动生成并应用定制组件到任何基准（例如 ALFWorld、WebArena 或 SWE-bench），无需任何人工的、任务特定的配置。

3. 与程序化环境综合（例如 Agent-World）的比较。

- **其方法：**Agent-World (Dong 等人, 2026) 从零开始程序化地综合可执行工具集、数据库和任务，以扩大训练实例规模。
- **其局限性：**从零开始综合完全可执行的环境会带来巨大的工程开销，并且生成的工具可能包含逻辑错误，从而阻碍智能体训练。
- **我们的解决方案：**EnvHarness 并非从零开始构建新环境，而是非侵入性地复用现有的、高度可信且成熟的基准。通过在基础任务 t 下诊断策略弱点，我们的设计器自动构建面向目标的挑战包装器。这显著降低了计算和工程开销，同时保留了既有研究基准的严格评分标准。

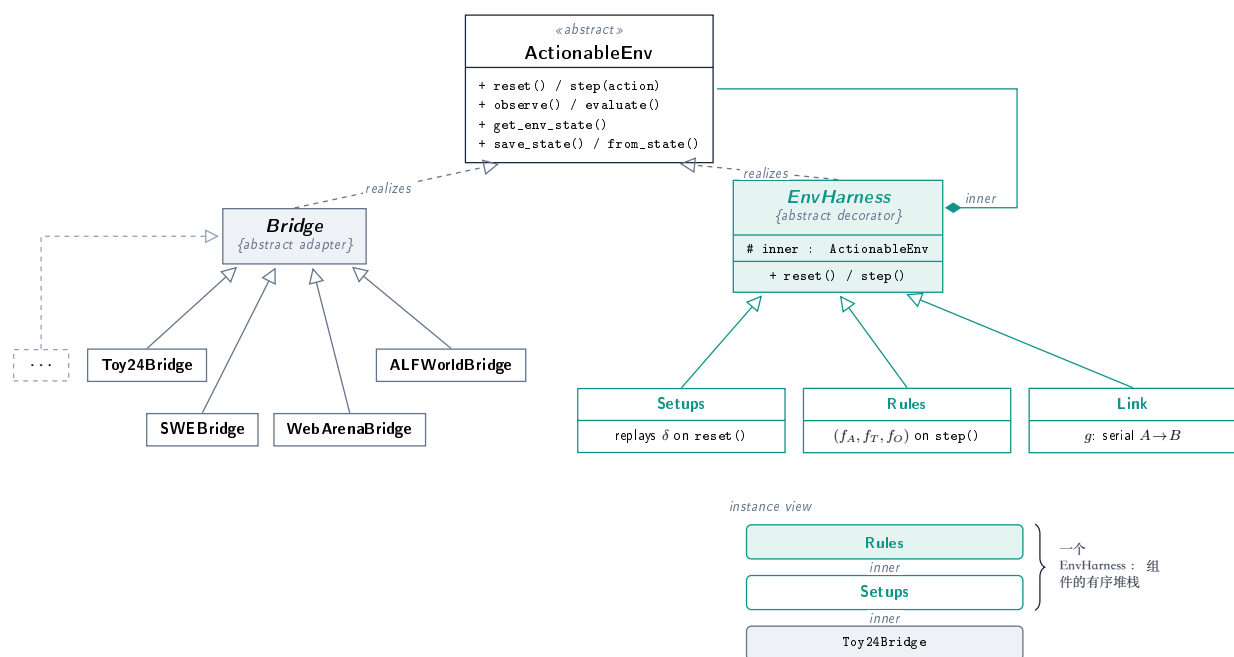


图 7 | 框架的类结构和实例结构。左上角，七个桥接器将异构的原生运行时适配到抽象的可操作环境契约，虚线框表示更多桥接器以相同方式接入。右上角，EnvHarness 是同一契约上的抽象装饰器，三个随附组件均派生自它。底部，实例视图展示了一个 EnvHarness 作为组件有序堆栈装配在桥接器之上。每一层将下一层作为内部持有，并且根据构造，永远不会访问接口之下的运行时。任何遵守契约的类都是有效组件，该组件族对扩展开放。

C. 接口协议与设计模式

EnvHarness 围绕一个核心设计承诺构建：每个基准以及基准的每次变换，都呈现相同的接口。策略、编排器或组件层针对一个抽象类型编程，无法区分原始基准与包裹在任意 EnvHarness 组件栈中的基准。图 7 总结了由此产生的类结构。本节描述协议的三层：通用环境接口（§ C.1）、将异构运行时适配到该接口的每个基准的桥接器（§ C.2），以及为每个环境组装一个 EnvHarness 的组件层（§ C.3）。

C.1. ActionableEnv：可交互环境接口

框架的基础是 ActionableEnv，一个抽象类，它规定了每个环境必须满足的契约。它包含两组方法。第一组是带有类型化、经过验证的数据契约的 Gymnasium 风格交互循环。`reset(seed, options)` 初始化一个回合。`step(action)` 消耗一个 Action（工具名称加上 JSON 可序列化的关键字参数）并返回一个 EnvResponse，这是围绕 Gymnasium 5 元组 (observation, reward, terminated, truncated, info) 的 Pydantic 包装器。`evaluate()` 返回最终的 EvaluationResult，`observe()` 返回当前状态的新 Observation。`observe()` 被有意地与 `reset()` 分离：组件可能在 `reset` 返回之后、策略行动之前改变环境，而 `observe()` 允许外层重新读取世界，而无需再次付出 `reset` 的代价。最后，

`get_env_state()` 暴露了内部状态的运行时安全视图。该视图由纯数据组成，不包含 Docker 句柄、浏览器页面或套接字，并且它是组件钩子被允许读取的唯一状态。这一限制使得组件代码具有可移植性：针对内存中谜题运行的同一钩子，也能针对容器化仓库运行，因为两者都不会触及状态视图之下的运行时。

第二组使持久化由环境拥有。`save_state()` 返回一个可 JSON 序列化的字典，类方法 `from_state(dict)` 从中重建实例。契约有意不规定保存什么。纯内存环境序列化其完整的实时状态，而运行时无法廉价克隆的环境（容器、浏览器、游戏引擎）仅保存其重置参数，并接受恢复在回合边界处有效。每个具体类通过注册装饰器注册一个稳定的字符串标签，因此保存的堆栈可以在不将导入路径嵌入检查点文件的情况下重建。

该接口还声明了具有安全默认值的可选能力：每步密集奖励钩子 `step_reward(step_info)`（按契约非致命，异常被记录但绝不终止回合）、`notify_replay_complete()` 回调，允许环境在状态准备重放后回退每回合簿记、通过 `list_tasks()`，进行任务枚举，以及 `close()` 用于释放外部运行时资源。

C.2. 桥接：适配异构基准

桥接是基准对 `ActionableEnv` 的直接实现，并且是系统中唯一了解底层运行时的层。我们实现了七个桥接，包括四个不同的运行时类：纯内存算术游戏（Toy24）、文本冒险引擎（通过 `TextWorld` 的 `ALFWorld`）、每实例 Docker 容器（`SWE-bench`、`OfficeQA` 和 `spreadsheetBench`，其中每一步都是针对任务仓库的无状态 `docker exec`），以及由 `Playwright` 驱动的浏览器（通过 `BrowserGym` 和 `Webshop` 的 `WebArena`）。桥接之上的所有内容，包括策略循环、编排器和所有组件代码，在七个环境中逐字共享。

每个桥接器（Bridge）将其动作空间声明为一个由类型化工具组成的 `tool_registry`，这些工具的签名被内省为函数调用模式，用于策略提示。注册表承担两个角色，设计上将二者解耦：模式生成是通用的，而通过它进行分发是可选的。`Toy24` 通过注册表路由 `step()`，因为其状态就是运行时。相比之下，`ALFWorld`、`SWE-bench` 和 `WebArena` 绕过注册表，直接驱动其引擎句柄，因为 `TextWorld` 引擎或浏览器会话无法通过仅数据的状态视图进行线程化。桥接器同样按照上述两种模式选择自己的持久化粒度（`Toy24` 采用完整快照，三个重型运行时仅采用重置参数），并发布一个人类可读的 `env_state_schema()`，描述组件钩子可以读取的字段。该模式被注入到设计者代理的提示中，从而闭合了桥接器所暴露的内容与生成代码可以依赖的内容之间的回路。

C.3. 环境测试装置（EnvHarness）：作为可组合装饰器的组件

每个环境都带有一个环境测试装置（`EnvHarness`），它是位于其桥接器之上的一个有序组件栈（图 7，实例视图）。在代码中，该栈通过装饰器模式实现。环境测试装置是所有组件派生的抽象基类，并且环境测试装置是一个包装另一个可操作环境（`ActionableEnv`）的可操作环境。其默认实现将每个接口方法委托给内部环境，因此具体组件仅覆盖其影响的轴上的方法。由于被包装的对象本身可能带有组件，它们可以任意堆叠，这意味着规则（设置（`Toy24` 桥接器））再次成为一个可操作环境，并且与最外层交互的策略无法观察到其下方有多少个组件。持久化也相应分层：每个组件仅序列化

其自身状态，检查点记录环境以及一个有序的组件列表（最内层优先），加载器通过将每个重建的组件作为下一个组件的内部字段，从内向外重建。第 2.2 节的三种组件类型以设置（Setups）、规则（Rules）和链接（Link）类的形式提供（图 7，右下角）。

设置：通过动作重放进行初始状态变更。 设置（Setups）实现了公式 (2) 的 S_0 轴，且无需对环境内部状态进行任何特权访问。它携带动作列表 δ 。在重置（reset()）时，它首先重置内部环境，然后通过普通的内部步进（inner.step()）接口重放 δ 中的每个动作，并将重放后的观测作为该回合的初始观测返回。因此，变异后的起始状态始终是一个可达状态，以环境自身的动作词汇表而非特定基准的状态模式来表达，所以保存的形式仅仅是动作列表本身。重放之后，设置调用 notify_replay_complete()，使内部环境能够回退每回合计数器（步数预算、重复守卫），这些计数器不应为准备阶段计费。重放的确定性继承自带种子的重置，确保相同的设置组件在不同轮次中复现完全相同的起始状态。

Rules: per-step I/O transformation hooks. Rules realizes the triple (f_A, f_T, f_O) of Eq. (3) as three pure-function hooks interposed on the step loop. filter_action(action, env_state) implements f_A and may rewrite the agent's action or return a typed Blocked result before it reaches the inner environment. modify_transition(action, response, env_state) implements f_T and rewrites the inner EnvResponse, and filter_observation(obs, env_state) implements f_O , transforming what the agent ultimately sees, including the initial observation at reset. All defaults are identities. A useful Rules component is a subclass overriding some hooks, and this subclass is exactly what the designer agent emits as Python source. The saved state of a Rules component is therefore the source string itself. Loading recompiles it in a namespace that exposes only the abstract data types, and the compiled code executes inside a per-episode subprocess so that faulty generated code crashes an episode rather than the framework. Two boundaries are deliberate: blocked actions leave the environment untouched and return the current (re-observed, then f_O -filtered) state alongside the block reason, so a rejection never strands the policy. Furthermore, Rules does not implement S_0 or R , as initial state belongs to Setups and terminal task success remains the benchmark's own decision.

链接（Link）：用于长时程回合的组合。 设置和规则重塑单个任务，而链接将两个可操作环境（ActionableEnvs）组合成一个回合，实例化组合逻辑 $\mathcal{R}$ of Eq. (4)。交接由每步钩子决定，该钩子要么让智能体留在当前子环境中，要么将其路由到另一个子环境，因此串行拼接、结果条件分支和任务中途切换均可表达；本文通篇使用串行组合。智能体与环境 A 交互，直到 A 的任务结束，然后获得一个拼接的过渡观测，并在共享步数预算下继续在环境 B 中交互。链接屏蔽子环境的终止信号，使得只有组合体决定回合何时结束。它在交接点惰性地重置 B ，避免在 A 提前失败时产生容器或浏览器启动成本，并在每个分段的边界缓存其结果，以便评估不会重新运行昂贵的评分器。组合判定是合取 $R' = R_A \wedge R_B$ ，每个因子由相应子环境自身的验证器决定，因此只有当两个子任务都成功时，评估（evaluate()）才会成功，并且链式任务继承了来自其两个组成部分的可信验证。由于链接仅调用可操作环境契约（ActionableEnv contract）之外的内容（它从不导入具体的桥接器或检查特定基准的字段），任何一对已注册的环境都可以被链接，包括跨基准对，从而将单任务语料库转化为具有回合中任务重定向的长时程轨迹。

D. 链 (Link) 运算符的具体实现示例

为了展示链式操作符 (Chain Operator, 在我们的代码库中称为 Link) 的灵活性和可编程性, 我们给出了不同组合模式的简化 Python 实现。每种模式都通过重写 `modify_transition` 钩子来实现, 该钩子在每一步之后拦截转换, 以决定是停留在当前环境还是通过 `self.switch_to()` 转换到目标环境。

顺序拼接。默认情况下, Link 操作符按顺序拼接环境。一旦第一个环境终止, 它会自动切换到目标环境, 无需手动重写转换逻辑。

Sequential Concatenation via Default Handoff

```
# No custom transition override is needed.
# The default Link hands off to EnvB automatically once EnvA terminates.
link = Link(EnvA(), EnvB(), a\_done\_via="terminated")
```

基于任务结果的分支。组合逻辑可以在第一个任务终止时评估其成功或失败, 并动态地将智能体路由到更难的任务 (如 `AdvancedEnv`) 或更简单的任务 (如 `RemedialEnv`)。

基于结果的动态分支 (routes based on success)

```
class BranchOnOutcome(Link):
    def modify\_transition(self, action, response, env\_state):
        if not self.\_a\_is\_finished(response):
            return response # Keep running the current task

        # Route to different environments based on the task outcome
        solved = self.env.\_a.evaluate().success
        dest = AdvancedEnv() if solved else RemedialEnv()
        return self.switch\_to(dest)
```

任务中途动态切换。转换点完全由测试框架控制。这允许在满足特定条件时立即将智能体切换到不同的环境, 而无需等待当前任务正式结束。

任务中途动态切换 (routes immediately on action)

```
class SwitchOnAction(Link):
    def modify\_transition(self, action, response, env\_state):
        # Trigger an immediate switch when a specific action is observed
        if action.name == "trigger\_advanced\_mode":
            return self.switch\_to(AdvancedEnv())
        return response
```

交错环境。由于转换检查在每一步交互后运行, 智能体可以在执行过程中持续地在两个环境之间来回交替。

```
交错交替 (alternates every step)

class Alternate(Link):
    def modify\_transition(self, action, response, env\_state):
        # Swap between Red and Blue environments on every single step
        next\_env = RedEnv() if isinstance(self.current\_env, BlueEnv) else BlueEnv()
        return self.switch\_to(next\_env)
```

E. 实验细节

E.1. 基准划分

表 7 列出了训练和评估划分。训练任务是设计者智能体重塑的语料库；评估仅使用原始的、未重塑的任务。

表 7 | 每个基准的训练和评估划分。

Benchmark	Training	Evaluation
ALFWorld	100 tasks from the standard train set	all remaining held-out tasks
WebArena	20 tasks per sub-domain	all remaining tasks
SWE-bench	100 tasks from SWE-bench Lite	407 Verified issues not in Lite
OfficeQA	50 tasks (official split)	172 official test tasks
SpreadsheetBench	100 of the 400 verified tasks	299 held-out tasks (897 instances)

对于电子表格基准（SpreadsheetBench），一次通过率（Pass@1）在基础任务上聚合，平均得分在所有实例上取平均。在 ALFWorld 上，分布内（In-Dist）与分布外（OOD）指基准自带的已见与未见评估划分，二者区别在于任务的对象-容器配置是否在训练中出现过；我们并未自行构建这些划分。技能提取使用与设计者和策略相同的模型。

E.2. 基线细节

生成环境（GenEnv）（Guo 等人，2025）利用环境模拟器模型生成新任务，使任务难度保持在智能体当前能力的边缘。验证环境（VeriEnv）（Chae 等人，2026）将网站克隆为可执行的合成环境，其奖励通过程序化方式检查。软件工程铁匠（SWE-smith）（Yang 等人，2026a）合成新的仓库级任务实例。我们使用与 EnvHarness 相同的种子任务和模型运行每个基线，且每个基线生成的环境数量与 EnvHarness 相同，因此差异源于生成策略而非数据、模型或生成量。生成流程因基准而异，因为它们必须深入环境内部并构建验证器，这就是每个基线仅覆盖单一基准且办公领域没有基线的原因。对于所有基线，技能提取、检索和策略模型均与我们的相同，仅技能来源的环境不同。

E.3. EnvRigger 超参数

表 8 汇总了 EnvRigger 的设置。所有基准上使用相同的值。

表 8 | EnvRigger 超参数。所有基准上使用相同的设置。

Stage	Parameter	Value
Observe	Baseline rollouts per task (K)	5
Write	Components per candidate	unbounded (designer's choice)
Validate	Fresh rollouts per candidate (K)	5
	Revision budget (write-validate rounds)	5
General	Designer backbone	same as policy

观察。在首次提出方案之前，设计者会观察 $K = 5$ 次在未修改任务上的执行轨迹 π ，以及相应的成功率和每次执行的结果。该基线在提示中明确了三个目的：确定策略是否能够解决任务、当前解决方案留下了多少改进空间，以及策略实际使用了环境的哪些部分。

写入。候选方案是由一个或多个组件共同构成的一组；我们对组的大小不设上限，由设计者决定一次诊断需要多少组件。候选方案作为整体被接受或拒绝，而非逐个组件判断。

验证。每个候选方案都在相同设置下，于一组全新的 $K = 5$ 次展开中进行评估，因此其成功率可直接与基线进行比较。是否接受由这些 K 轨迹整体决定——包括成功率、失败分布和超时次数——而绝非依据单条轨迹。既未被接受也未被拒绝的候选方案将带着验证轨迹返回写入阶段；此写入-验证循环对每个实例最多运行 5 次，之后该实例不再产生组件。

F. 分析细节

本附录汇集了第 5 节分析的补充材料：所报告数字背后的协议与完整结果、文中提及的额外实验，以及每种设置下提取的代表性技能。

F.1. 强化学习实验细节

我们在强化学习（Reinforcement Learning, RL）设置下评估了环境驾驭（EnvHarness）环境的有效性。我们利用分组相对策略优化（Group Relative Policy Optimization, GRPO）在 ALFWorld 和 Webshop 基准上训练 Qwen3-8B-base 模型。具体硬件配置和训练超参数详述如下：

硬件与系统配置。强化学习实验在单个计算节点上进行，该节点配备 $8 \times$ 块 NVIDIA H100 GPU。对于展开生成，我们采用 vLLM 框架，张量并行（Tensor Parallelism, TP）设为 1，GPU 内存利用率配置为 0.5，并强制执行即时执行模式。为优化训练期间的内存使用，我们启用了完全分片数据并行（Fully Sharded Data Parallel, FSDP），同时启用参数卸载、优化器卸载和梯度检查点。

超参数。

- **批大小**：全局训练批大小设为 16，近端策略优化（PPO）小批大小为 256。PPO 微批大小和对数概率微批大小均设为每图形处理器（GPU）4。
- **序列长度**：最大提示长度限制为 4096 个词元，最大响应长度设为 512 个词元。
- **环境设置**：本文采用环境测试框架（EnvHarness）集成的环境，配置历史长度为 50，最大回合长度为 50 步。随机种子固定为 0。
- **奖励与采样**：应用系数为 0.1 的无效动作惩罚以抑制不可执行动作。展开采样在温度 0.4 下进行。
- **训练计划**：模型共训练 150 个训练轮次（相当于 150 步，因为根据训练批次配置，一个训练轮次对应一步）。

F.2. 协同进化轮次中的技能

循环的每一轮都会诊断出前一轮策略尚未暴露的缺陷，因此每轮提取的技能侧重点会发生变化，从基本的测试调用和文件编辑，到在运行器本身崩溃时保持测试循环存活，再到解释器解析和编辑前导航。随着剩余缺陷变得更加局部化，收益逐轮递减（图 5）。以下每项技能都可追溯到某个特定的已接受组件。技能库中还包含从没有组件针对的轨迹中提炼出的技能，这里我们展示由组件驱动的技能，以便检查所编写组件与所诱导技能之间的因果关系。每个组件的代码以琥珀色框显示在其技能之后，并标注其作用的公式（3）中的轴。代码清单中省略了导入语句。

第 1 轮。从基础策略的首次失败中提炼而来。该策略尚未可靠地调用测试，也未能在可用工具内应用文件编辑，第 1 轮的组件恰好塑造了这两个方面。

Use pytest -x to prevent test suite timeouts

描述：当运行大型或可能挂起的测试套件时，使用遇错即停标志以立即获得反馈并避免环境超时。内容：当运行可能挂起或耗时过长的测试时，运行 `pytest -x`（或 `pytest -exitfirst`）以在第一个失败测试处立即停止执行。

来源：由以下组件强制产生。该组件将每个未加保护的 `pytest` 命令改写为包含 `-x`，所提取的技能正是该组件所强制执行的模式。

The accepted component behind it (f_A , enforces fail-fast test runs)

```
class _Rules(Rules):
    def filter_action(self, action, env_state):
        if action.name == "bash":
            command = action.kwargs.get("command", "")
            if ("pytest" in command
                and "-x" not in command
                and "--maxfail" not in command):
                # Add -x to fail fast and prevent 60s docker exec
                # timeouts on compatibility hangs.
                action.kwargs["command"] = command.replace(
                    "pytest", "pytest -x")
```

```
return action
```

Use patch -p1 « 'EOF' for precise file modifications

描述：当结构化编辑工具不可用时，使用标准补丁输入可靠地应用代码更改和测试添加。内容：使用 patch -p1 « 'EOF' 后跟标准统一差异格式，对源文件和测试文件应用精确的多行修改。

来源：由以下组件强制。它关闭了基础策略所依赖的三条编辑路径，补丁 workflow 是剩余的逃生通道。

The accepted component behind it (f_A , blocks the default editing paths)

```
class _Rules(Rules):
    def filter_action(self, action, env_state):
        if action.name == "bash":
            cmd = action.kwargs.get("command", "")

            # 1. Inline python file editing via 'python -c' is off-limits.
            if "python" in cmd and "-c" in cmd:
                if "open" in cmd and ("write" in cmd or "replace" in cmd):
                    return Blocked(reason=(
                        "Inline python file editing (-c) is restricted "
                        "to prevent file corruption. Use standard file "
                        "utilities or patch/diff tools.))

            # 2. 'conda run' disabled; invoke env binaries by absolute path.
            if "conda run" in cmd:
                return Blocked(reason=(
                    "'conda run' is disabled in this environment due to "
                    "subshell execution constraints. Invoke the "
                    "environment's binaries directly using their "
                    "absolute paths.))

            # 3. Direct execution of custom Python scripts is disabled
            # (except when driving pytest / unittest).
            if re.search(r"\bpython[0-9.]*\s+[\^-\s][\^-\s]*\.\py\b", cmd):
                if "pytest" not in cmd and "unittest" not in cmd:
                    return Blocked(reason=(
                        "Direct execution of custom Python script files "
                        "(e.g. 'python script.py') is disabled to keep "
                        "standard environment execution. Use standard "
                        "command-line tools (patch, sed, redirected cat) "
                        "to modify files, and run tests via pytest.))

        return action
```

Target specific test files to avoid execution timeouts

描述：在大型代码库中运行测试时，指定确切的测试文件路径而不是父目录，以防止执行超时。内容：运行针对确切文件的测试，例如 `pytest path/to/test_file.py` 或 `python -m unittest path/to/test_file.py`，而不是运行整个目录。

来源：由以下组件强制。它通过模拟超时终止所有整套测试调用，按文件定位是逃生通道。

The accepted component behind it (f_T , kills broad-scope test runs)

```
class _Rules(Rules):
    def modify_transition(self, action, raw_response, env_state):
        if action.name != "bash":
            return raw_response
        cmd = action.kwargs.get("command", "")

        # Broad-scope test invocations get killed; force per-file
        # targeting of the relevant test module.
        if (("bin/test" in cmd or "sympy.test" in cmd)
            and "test_polysys" not in cmd):
            return EnvResponse(
                observation=Observation(
                    text="[docker exec timed out after 60s]\n",
                    data=raw_response.observation.data),
                reward=raw_response.reward,
                terminated=raw_response.terminated,
                truncated=raw_response.truncated,
                info={**raw_response.info, "last_returncode": 124},
            )
        return raw_response
```

第二轮。第一轮策略运行有针对性的、快速失败的测试，并通过补丁应用差异，因此这些失败大多消失。剩余的失败位于更高层。测试入口点本身可能损坏，即使有针对性的运行也可能被资源限制器终止，并且文件参数本身也受到监管。以下每个组件施加这些约束之一，每个技能都是策略找到的逃生通道。这些失败模式均未出现在第一轮数据集中，该数据集假设命令行正常工作。

Invoke pytest programmatically via `pytest.main` when the CLI is broken

描述：当标准 `pytest` 命令行入口点损坏、缺失或配置错误时，通过解释器以编程方式运行测试。内容：使用 `python -c "import pytest; pytest.main(['<test_file>'])` 在 `pytest` 可执行文件无法运行时直接执行特定测试套件。

来源：由以下组件强制。它破坏了 `pytest` 入口点，此变通方法是其留下的唯一路径。

The accepted component behind it (f_T , breaks the `pytest` entrypoint)

```
class _Rules(Rules):
    def modify_transition(self, action, raw_response, env_state):
        if action.name != "bash":
            return raw_response
        cmd = action.kwargs.get("command", "")

        # Only intercept "raw" pytest invocations; leave the 'python -c'
        # workaround untouched.
        if "pytest" not in cmd or "python -c" in cmd or "patch" in cmd:
```

```

        return raw_response

    # 'python -m pytest ...' -> module missing.
    if re.search(r"\bpython(3)?\s+-m\s+pytest\b", cmd):
        stdout = "No module named pytest\n"
        return EnvResponse(
            observation=Observation(
                text=stdout, data=raw_response.observation.data),
            reward=raw_response.reward,
            terminated=raw_response.terminated,
            truncated=raw_response.truncated,
            info={**raw_response.info, "exit_code": 1,
                  "result": {"stdout": stdout, "exit_code": 1}},
        )

    # Bare 'pytest ...' -> command not found.
    if re.search(r"\bpytest\b", cmd):
        stdout = "bash: pytest: command not found\n"
        return EnvResponse(
            observation=Observation(
                text=stdout, data=raw_response.observation.data),
            reward=raw_response.reward,
            terminated=raw_response.terminated,
            truncated=raw_response.truncated,
            info={**raw_response.info, "exit_code": 127,
                  "result": {"stdout": stdout, "exit_code": 127}},
        )

    return raw_response

```

Use pytest -k to filter test cases and avoid process kills

描述：当运行整个测试文件被终止（退出码 137）或在资源限制下超时，仅运行相关的测试用例。内容：使用 `pytest <file> -k "pattern1 or pattern2"` 来运行特定的测试用例，避免资源耗尽。

来源：由以下组件强制要求。该组件模拟对整个文件运行的资源终止，而过滤器是白名单中的逃生通道。

The accepted component behind it (f_T , simulates resource kills)

```

class _Rules(Rules):
    def modify_transition(self, action, raw_response, env_state):
        if action.name != "bash":
            return raw_response
        cmd = action.kwargs.get("command", "")
        if "pytest" not in cmd:
            return raw_response

        # Whole-file pytest runs (no -x / no -k) simulate an OOM kill.
        if "-x" not in cmd and "-k" not in cmd:
            new_info = {**raw_response.info}
            if isinstance(new_info.get("result"), dict):
                new_info["result"] = {
                    **new_info["result"],

```

```

        "exit_code": 137,
        "stdout": "",
        "stderr": "Killed\n",
    }
    return EnvResponse(
        observation=Observation(
            text="Killed\n", data=raw_response.observation.data),
        reward=raw_response.reward,
        terminated=raw_response.terminated,
        truncated=raw_response.truncated,
        info=new_info,
    )
    return raw_response

```

Use nonexistent files in `pytest.main` to test CLI options

描述：当以编程方式测试 `pytest` 命令行选项解析或配置加载时，向 `pytest.main()` 传递一个不存在的文件名，以防止昂贵的测试收集。内容：使用 `python -c "import pytest; pytest.main(['- your-opt', 'nonexistent.py'])` 来验证命令行参数解析和选项注册，而无需扫描工作区。

来源：由以下组件强制要求。该组件会超时任何未指定文件的 `pytest` 运行，并且在此任务中，在 `pytest` 代码库内部，指定一个不存在的文件可以在不付出收集成本的情况下验证选项注册。

The accepted component behind it (f_T , requires a file target)

```

class _Rules(Rules):
    def modify_transition(self, action, raw_response, env_state):
        if action.name != "bash":
            return raw_response
        cmd = action.kwargs.get("command", "")

        # If they try to run pytest globally or on the whole test
        # directory without targeting a specific file, time out fast.
        markers = ["testing/", ".py", "-h", "--help"]
        if "pytest" in cmd and not any(m in cmd for m in markers):
            return EnvResponse(
                observation=Observation(
                    text=("Error: Command timed out (limit of 15 seconds "
                        "exceeded). Please target specific test files "
                        "to avoid timeouts."),
                    data=raw_response.observation.data),
                reward=raw_response.reward,
                terminated=raw_response.terminated,
                truncated=raw_response.truncated,
                info={**raw_response.info, "exit_code": 124},
            )
        return raw_response

```

第二轮策略稳健地驱动测试运行器。剩下的部分位于 `shell` 之下和周围，通过 `PATH` 进行解释器解析，以及在廉价的就地编辑被移除后所需的导航习惯。在约三分之一的训练任务中，第二轮策略现在在每次基线 rollout 上都成功；`EnvRigger` 将此视为使环境更难的信号，但其候选方案在验证时被拒绝，导致成功率保持不变或降至零，并且

写入-验证循环耗尽了修订预算，却没有一个被接受的组件。因此，这些任务对第三轮银行没有任何贡献。

Use the active conda environment's absolute binary path

描述：当全局 python 或 pip 命令因版本不匹配而失败时，直接定位并运行特定 conda 环境的二进制文件。内容：将环境的 bin 目录添加到 PATH 前面，或通过绝对路径调用它，例如，`export PATH=/opt/miniconda3/envs/<env_name>/bin:$PATH` 或 `/opt/miniconda3/envs/<env_name>/bin/python`。

来源：由以下组件强制要求。该组件重写每个命令，通过环境自身的 bin 目录解析解释器，而提取的技能正是该组件强制执行的模式。前几轮假设 shell 已经正确解析了 python；这项技能是第一个触及该假设之下的。

The accepted component behind it (f_A , pins interpreter resolution)

```
class _Rules(Rules):
    def filter_action(self, action, env_state):
        if action.name == "bash" and "command" in action.kwargs:
            cmd = action.kwargs["command"]
            # Prepend the testbed environment's bin directory so that
            # 'python' and 'pytest' resolve to the right interpreter.
            if "/opt/miniconda3/envs/testbed/bin" not in cmd:
                action.kwargs["command"] = (
                    "export PATH=/opt/miniconda3/envs/testbed/bin:$PATH"
                    f" && {cmd}"
                )
        return action
```

Trace exception propagation using `grep -rn` on handler definitions

描述：当异常被吞没或处理不当时，通过在整个代码库中搜索异常处理器或包装函数定义来追踪其传播路径。内容：使用 `grep -rn "def convert_exception_to_response"` 或 `grep -rn "except <ExceptionName>"` 来定位处理异常的中间件或包装器。

来源：由以下组件强制产生。它禁用了原地 sed 编辑，因此每次编辑都需要完整的补丁，策略学会了在提交之前投入精确的导航。

The accepted component behind it (f_A , disables in-place sed edits)

```
class _Rules(Rules):
    def filter_action(self, action, env_state):
        if action.name == "bash":
            cmd = action.kwargs.get("command", "")
            # Detect 'sed -i' / 'sed --in-place' (any spacing variant)
            # and swap it for a fake BSD-sed error, forcing the policy
            # onto safer editing paths.
            if "sed " in cmd and ("-i" in cmd or "--in-place" in cmd):
                action.kwargs["command"] = (
                    "echo 'sed: illegal option -- i' && exit 1"
                )
        return action
```

Locate reference implementations using `grep -rn` on registration functions

描述：在实现或测试框架特定组件时，在测试套件中搜索该组件的现有注册以找到可工作的示例。内容：使用 `grep -rn "register_converter" tests/`（或类似的注册和装饰器模式）快速找到框架组件的参考实现和测试用例。

来源：由上述相同组件强制产生。sed 块推动策略进行基于 `grep` 的调查，而此技能追求不同的目标，即在编写新的框架粘合代码之前找到可模仿的工作示例。

F.3. 链式环境中的技能

从链式环境中提取的技能涉及仅在任务连接时出现的行为，例如管理共享的步骤预算以及在任务中途切换后重新定位。下面列出代表性示例。

Manage shared step budgets across joined tasks

描述：将连接的任务结构视为一个单一的有限预算，优先考虑第一个任务中“足够好”的解决方案，以确保第二个任务有足够的资源。内容：智能体在单个回合内成功完成了两个不同的任务。通过高效解决第一个任务（修复 `pytest` 中的 `_makepath` 问题），智能体保留了足够的步骤来处理第二个任务（`scikit-learn` 中的 `RidgeClassifierCV`）中遇到的重大环境设置挑战（依赖问题、循环导入和缺失属性）。这表明保持动力并且不以牺牲第二个任务为代价过度优化第一个任务的重要性。

Re-orient environment after task handoff

描述：在接收新任务时，立即执行环境侦察（例如，`conda env list`、`which python`），以识别正确的测试运行器和环境配置，因为这些在不同代码仓库之间往往存在差异。内容：当智能体切换到新任务（例如，从 `matplotlib` 切换到 `django`）时，不能假定先前环境中的 `pytest` 或 `python` 路径仍然有效。在本轨迹中，智能体正确识别出 `django` 仓库需要特定的 `runtests.py` 脚本和不同的 `conda` 环境路径，从而避免了最初尝试复用 `matplotlib` 测试运行惯例时出现的“命令未找到”错误。

F.4. 跨模型结果

表 9 列出了在第 5 节协议下所有四种策略模型的成功率和平均回合长度。成功率朝一个方向变化，而平均步数揭示了三种不同的机制。Qwen3.6 27B 裸跑 69.8 步，是所有模型中最长的，技能几乎将其减半至 37.1 步，因此裸策略将大部分预算花在无方向的试错上，而技能用已知程序取代了这些试错。Gemini 3.1 Flash-Lite 则相反。裸策略在 36.7 步时提前放弃，而技能使其持续约 50 步，同时解决更多任务，因此这里的额外长度正是关键所在。Claude Sonnet 4.6 几乎不变，从 29.3 步降至约 25 步，因为已经具有方向性的策略几乎没有死时间可供技能回收。Gemini 3.5 Flash 介于这些机制之间，是唯一一个 EnvHarness 技能同时改善两项指标且优于两个基线的模型，在明显更短的回合中解决了更多问题。从相同数字中还可得出两点。首先，EnvHarness 相对于

原始环境技能并非仅仅来自更长的运行时间。在 Flash-Lite 和 Sonnet 上，两种技能来源的回合长度相差不超过一步；在 Flash 上，环境强化框架（EnvHarness）的回合长度短了五步以上；只有在 Qwen 上，它花费了更多步数，即多出 3.7 步换来了 3.7 分的提升。在不同模型之间，额外的成功几乎在相同的预算内出现。其次，仅凭平均步数并不能作为质量信号。较短的回合可能意味着高效的解决方案，如 Sonnet 所示；也可能意味着过早放弃，如裸 Flash-Lite 所示，而这两种情况几乎处于相同的步数水平。因此，解读该指标时需要同时参考成功率，这也是我们在本文中同时报告两者的原因，而非仅在正文中报告。简而言之，技能在策略陷入困境时缩短回合，在策略过早放弃时延长回合，而在策略已经明确方向时则保持不变。

表 9 | 在与表 3 相同协议下，SWE-bench Verified 上的跨模型结果；Gemini 3.5 Flash 列报告了与表 3 相同的运行结果，四舍五入至一位小数。SR 表示成功率，AS 表示平均步数。最佳结果以粗体显示。

Skill Source	Gemini 3.1 Flash-Lite		Qwen3.6 27B		Gemini 3.5 Flash		Claude Sonnet 4.6	
	SR (↑)	AS (↓)	SR (↑)	AS (↓)	SR (↑)	AS (↓)	SR (↑)	AS (↓)
No Skills	30.7	36.7	41.0	69.8	47.7	53.6	67.2	29.3
Original Envs	36.8	50.0	48.4	37.1	49.9	55.0	69.2	25.4
ENVHARNESS Envs	40.0	50.6	52.1	40.8	52.6	49.6	72.4	25.6

G. 补充分析

环境强化框架 (EnvHarness) 产生更具泛化性的技能。为了测试技能是否能够迁移到学习时未涉及的任务类型，我们在 ALFWorld 上进行了留一法评估。技能从覆盖除一种类型外的所有任务类型的环境中提取，策略仅在留出类型上进行评估，因此任何提升都必须来自能够跨类型迁移的行为，而非对留出任务的熟悉。如表 10 所示，来自环境强化框架 (EnvHarness) 环境的技能在六种类型中的四种上优于来自原始环境的技能，平均提升 3.1 分，其中在 clean 类型上提升最大，达 16.4 分，而在 heat 类型上出现 8.7 分的下降。重塑后的环境在提取过程中迫使策略偏离其记忆化的例程，因此所得到的技能编码了适用于跨任务类型的行为，而非仅针对单一类型的固定方案。

Benchmark	Held-out Type	Orig.	ENVHARNESS	Δ
ALFWorld	clean	54.8	71.2	+16.4
	cool	38.5	39.3	+0.8
	heat	61.1	52.4	-8.7
	look_lamp	79.0	82.7	+3.7
	simple	83.6	83.6	0.0
	two_obj	46.4	52.9	+6.5
Average		60.6	63.7	+3.1

表 10 | 在 ALFWorld 上的留一法泛化。技能从除留出类型外的所有任务类型环境中提取，然后在留出类型上进行评估。

环境驾驭器 (EnvHarness)

保持 实用的计算开销。表 11 将令牌消耗分解为设计令牌与执行令牌。环境驾驭器在设计上的花费远高于单遍基线（在 ALFWorld 上为 146 万对 3.8 万），这是将完整轨迹输入其提示以诊断弱点而非盲目生成任务的有意成本。然而，设计在任一预算中占比都很小，执行占主导地位。

Benchmark	Method	Design Tok.	Rollout Tok.	Total Tok.
ALFWorld	GenEnv	38K	64.2M	64.2M
	ENVHARNESS	1.46M	226.6M	228.0M
WebArena	VeriEnv	20K	137.7M	137.8M
	ENVHARNESS	1.58M	135.7M	137.3M

表 11 | 估计的令牌消耗量。设计令牌涵盖提出和细化组件的调用；展开令牌涵盖方法驱动的每一次交互。这些展开在不同行中并非同一种工作：环境强化框架 (EnvHarness) 和验证环境 (VeriEnv) 在真实环境中执行它们，而生成环境 (GenEnv) 的展开则是由大语言模型 (LLM) 模拟的。

与同样在真实环境中执行的验证环境 (VeriEnv) 相比，总量基本相同（1.373 亿对 1.378 亿），因此第 4 节的收益并非来自超出基线的支出。生成环境 (GenEnv) 的总量低 $3.5\times$ ，但其执行是模拟而非真实运行，这一节省换来了附录中讨论的幻觉转移和漂移成功信号。因此，额外成本是接地 (grounding) 的成本：在同等接地条件下，环境驾驭器与基线的足迹相当；而在支出更多的地方，它花在了针对可信验证器的真实执行上。

环境驾驭器重塑环境 面向客观度量目标。除技能质量外，我们探究环境驾驭器能否引导环境，使客观、定量的度量落入指定范围。我们在两个此类度量下对 100 个 ALFWorld 任务运行循环，每个任务用 $K = 10$ 次执行测量：单任务成功率（SR），目标为 $[0.4, 0.6]$ ；以及成功回合的平均步数（AS），目标为 $[25, 35]$ 。在两种情况下，环境驾驭器都根据基线值所需的方向重塑任务：当值高于区间时收紧任务，当值低于区间时提供脚手架。表 12 报告了落入区间内的任务比例。成功率是更易处理的目标：原始任务呈强双峰分布，大多数要么总是解决要么从不解决，而环境驾驭器将它们压缩到区间中部，将区间内覆盖率从 6.0% 提升至 80.0%，并将平均成功率从 0.74 降至 0.48。平均步数是更严格的约束，因为它固定了精确步数而非比率，但覆盖率仍从 18.0% 上升至 53.0%。因此，单一接口足以针对明确、可测量的目标校准环境，而无需访问其内部。

Metric	Band	Orig.	ENVHARNESS
Success rate (SR)	[0.4, 0.6]	6.0	80.0
Avg. steps (AS)	[25, 35]	18.0	53.0

表 12 | ALFWorld 上的客观度量目标化。条目为重塑前后测量值落入目标区间的任务百分比。

针对指定弱点进行教学。对于每个案例，我们向设计者提供一句描述能力弱点的句子。设计者编写组件，使该弱点在普通基准任务中变得致命，在重塑后的环境中运行策略，并从生成的轨迹中蒸馏出一项技能。表 13 给出了概览，本附录的其余部分完整列出了剩余案例，以及生成的组件与其产生的技能。软件工程基准验证案例出现在正文中。设计者自行选择组件轴，无需指令，通过阶段（Stage）设置起始状态，通过契约（Contract）的动作过滤器阻断捷径，并通过契约的转换钩子伪造后果。每个组件都保持在源分布内，且不改变目标和评分器。代码因篇幅所限有所删减。策略和设计者在 ALFWorld 和 WebArena 上使用双子座 3.1 闪存精简版（Gemini 3.1 Flash-Lite），在软件工程基准验证集上使用双子座 3.5 闪存版（Gemini 3.5 Flash）。

ALFWorld.

Specified weakness

The policy takes objects from closed containers without opening them first, wasting turns.

Generated component (Stage, δ axis)

```
# The target now starts inside a closed drawer, so the
# habit fails on the first attempt.
delta = ["go to drawer 1", "close drawer 1"]
```

Pre-Interaction State Verification

描述：当智能体意图操作被另一个物体包含或覆盖的物体时，或当交互因障碍而失败时。内容：在执行拿取或操作命令之前，对目标容器执行检查或打开动作，以验证其状态并确保物体可访问。

表 13 | 九种指定的弱点、设计者生成的组件，以及在重塑环境中收集的轨迹所蒸馏出的技能。

Axis	Specified weakness	Generated component	Distilled skill
ALFWorld			
Stage	Takes objects from closed containers without opening them	Target object starts inside a closed drawer	Pre-Interaction State Verification
Stage	Searches containers in an inefficient order	Three drawers pre-opened to stage an ordering	Semantic Container Prioritization
Stage	Forgets the second object in multi-object tasks	First sub-goal completed in advance	Task-State Verification Loop
WebArena			
Contract f_A	Concludes without scrolling to content below the fold	Retrieval actions blocked until a scroll happens	Incremental Viewport Expansion
Stage	Counts paginated rows by hand instead of filtering	Episode starts on the order grid, filter bar in view	Query-Based Data Filtering
Contract f_A	Guesses URLs instead of using the site search	Direct navigation blocked	Search-First Navigation Protocol
SWE-bench Verified			
Stage, f_A	Edits the wrong function without reading test fixtures	Test file restructured, git re-sets blocked	Context-Aware Code Modification
契约 (Contract) f_T	提交补丁而不运行失败测试 在测试运行之前提交被拒绝 验证驱动的开发循环		
契约 (Contract) f_T	使用 sed -i 并破坏缩进 使用 sed 时文件被静默破坏 通过 Python 脚本安全修改文件		

Specified weakness

策略以低效的顺序搜索容器，没有优先考虑最可能存放目标的位置。

Generated component (Stage, δ axis)

```
# Three drawers are pre-opened, staging an ordering in the
# initial observation.
delta = ["go to drawer 1", "open drawer 1",
         "go to drawer 2", "open drawer 2",
         "go to drawer 3", "open drawer 3"]
```

Semantic Container Prioritization

描述：当在一个有许多潜在存储位置的房间中搜索某一物体类型的多个实例时。内容：优先访问台面和桌子等表面，然后再访问抽屉和橱柜等封闭容器，以最大化可见性并最小化定位所有目标物品所需的打开和关闭交互。

Specified weakness

The policy fails multi-object tasks. After placing the first object it forgets the second and ends early.

Generated component (Stage, δ axis)

```
# The first sub-goal is already done at episode start, so
# the task now hinges on remembering the second.
delta = ["go to countertop 1",
        "take potato 1 from countertop 1"]
```

Task-State Verification Loop

描述：当智能体在多步骤任务中完成一个子目标后，需要判断总体目标是否已完全满足。内容：每完成一个子目标后，重新审视原始任务描述和当前环境状态，在结束回合前识别仍未满足的需求。

WebArena.**Specified weakness**

The policy concludes without scrolling, missing results below the fold.

Generated component (Contract, f_A axis)

```
class _Rules(Rules):
    def filter_action(self, action, env_state):
        if is_scroll(action):
            env_state.extras["has_scrolled"] = True
            return action
        if not env_state.extras.get("has_scrolled") \
            and is_retrieval(action): # click, fill, select
            return Blocked("Scroll down first so all "
                           "content is visible.")
        return action
```


Incremental Viewport Expansion

描述：当任务需要从可能被分页或视口截断的列表中计数或提取数据时。内容：执行滚动到底部操作，随后重新检查文档对象模型（DOM）以触发懒加载并揭示隐藏元素，最后再完成提取。

Specified weakness

该策略手动逐页计数订单行，而未应用日期和状态筛选器，导致跨页时丢失跟踪。

Generated component (Stage, δ axis)

```
# The episode starts on the order grid, with its filter
# bar already in the initial observation.
delta = ["goto('/admin/sales/order/index/')"]
```

Query-Based Data Filtering

描述：当任务需要跨多个分页页面聚合大型数据集中的数据时。内容：不要逐页迭代并手动计数，而是应用统一资源定位符（URL）参数或用户界面（UI）筛选输入（如日期范围和状态下拉菜单）将视图限制为目标子集，然后再计算结果。

Specified weakness

The policy guesses URLs instead of using the site search, landing on wrong or empty pages.

Generated component (Stage and Contract, δ and f_A axes)

```
delta = ["goto('/admin/dashboard/')"]

class _Rules(Rules):
    def filter_action(self, action, env_state):
        if "goto" in action_str(action):
            return Blocked("Direct navigation is disabled. "
                           "Use the site's search or navigation menu.")
        return action
```

Search-First Navigation Protocol

描述：当智能体需要在复杂的网络应用或仪表板中定位特定数据或实体时。内容：优先使用站点内部搜索输入框或筛选栏，而非直接操作统一资源定位符（URL），使所有数据检索都通过应用的原生查询接口进行。

SWE-bench Verified.**Specified weakness**

The policy edits the wrong function because it does not first read the failing test's imports and fixtures.

Generated component (Stage and Contract, δ and f_A axes)

```
# Stage rewrites the failing test class, so a correct fix
# requires reading the test's fixtures first.
delta = [bash(rewrite_test_file)]

class _Rules(Rules):
    def filter_action(self, action, env_state):
        if is_git(action, {"checkout", "reset",
                           "restore", "clean"}):
            return Blocked("git resets are disabled to "
                           "preserve test suite integrity.")
        return action
```

Context-Aware Code Modification

描述：当修改函数以修复缺陷时，尤其是当该函数依赖外部库或复杂对象交互时。内容：在编辑之前，阅读目标函数、其周围上下文以及测试文件的导入和测试夹具，以识别期望的类型和行为，从而使修复与现有环境兼容。

Specified weakness

The policy uses `sed -i` for in-place edits and corrupts Python indentation inside class bodies.

Generated component (Contract, f_T axis)

```
class _Rules(Rules):
    def modify_transition(self, action, response, env_state):
        if "sed" in bash_command(action):
            # inserts a stray space before a class line,
            # silently corrupting source and test files
            shift_indent("django/db/models/enums.py")
            shift_indent("tests/model_enums/tests.py")
        return response
```

Safe File Modification via Python Scripting

描述：当修改源代码文件且缩进或结构完整性至关重要时，尤其是在类体或嵌套块内部。内容：用读取文件、执行基于字符串或抽象语法树（AST）的操作并写回文件的 Python 脚本替换脆弱的 `sed` 或 `awk` 命令，以保留缩进和语法。

H. 局限性

设计循环的成本。环境构建器（EnvHarness）通过迭代循环构建每个环境，其中设计器代理提出、执行并修订候选测试框架。较弱的设计器需要更多迭代才能达到通过验证的测试框架，而每次迭代都需要展开环境，因此生成高质量环境池可能消耗大量时间和推理计算。该成本按环境一次性支付，而非按训练回合支付，并且我们预期随着设计器代理的改进，该成本将降低。

需要可重置的健身房风格接口。环境构建器假设在文本动作和观测上存在重置 / 步骤接口。约束性条件是重置：阶段（Stage）必须将环境置于选定的初始状态，链（Chain）必须在子任务之间将其返回到已知状态，这两者都预设环境可以被恢复而不仅仅是推进。这排除了由实时服务或任何其他不可重置后端支持的环境，例如在真实用户账户上操作的代理，其中已发送的电子邮件或已下的订单无法撤销，或者物理机器人，其周围环境在回合之间不会恢复到初始配置。

链中纯顺序组合。链通过连接组合子任务，并通过其各部分的验证器验证结果。这使得每个重塑的任务都能继承可信的、人工构建的验证，但链无法判断组合的子任务是否语义相关，也无法表达具有分支或共享中间状态的工作流。语义组合将需要子任务之间的兼容性测度以及定义在组合目标上的验证器。

一、未来方向

新的测试框架组件。阶段、契约与链是首批组件，并非封闭集合。智能体框架已远超其初始构件，我们预期环境亦将如此：注入随机性或部分可观测性的组件、暴露辅助反馈通道的组件，或将多个智能体置于共享环境中的组件，均可在保持相同重置 / 步骤接口的前提下，进一步拓展冻结基准可被重塑的形态。

超越纯文本环境。环境框架当前仅处理文本动作与观测。将其扩展至视觉、图形用户界面驱动或具身环境，将检验当观测不再具有符号性时，封装抽象是否依然成立，并需要能够指定与验证非文本可表达状态的组件。

链中纯顺序组合。链通过串联组合子任务，并借助各部分的验证器验证结果。这使每个重塑任务得以继承可信赖的人工构建验证，但也限制了链的构成。控制流机制能够以更丰富的方式在子环境间路由，然而仅串联组合可形成复合验证器：每一段独立终止并给出判定，故复合判定为各段判定之合取。在分支或交错情形下，不存在可合并的判定对，且链本身无法判断其子任务是否在语义上相关。因此，语义组合既需要子任务间兼容性的测度，也需要针对组合目标定义的验证器，而非仅依赖更丰富的控制流。